

**STYLE SYNTHESIS: Bridging Art and Technology through Image
Transfer**

**A Project Work Report submitted to the Jawaharlal Nehru Technological
University Kakinada in Partial Fulfillment of the Requirements for the Award
of the Degree**

**of
BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY**

Submitted by

**P.SRINITHI
(20501A1293)**

**M.SRIRAM
(20501A12B1)**

**E.SANJANA
(20501A12C5)**

**P.RISHITHA
(20501A1293)**

**Under the guidance of
Mr.Ch.Praneeth,M.Tech,(Ph.D)
Assistant Professor**



Department of Information Technology

PRASAD V POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY

(Permanent Affiliated to JNTUK: Kakinada, Approved by AICTE)

**(An NBA (all UG programs) and NAAC - A+ accredited and ISO 9001:2015 certified
Institution)**

Kanuru, Vijayawada – 520007

April,2024

PRASAD V POTLURI

SIDDHARTHA INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)(Affiliated to JNTU: Kakinada, Approved by AICTE)

(An ISO Certified Institution and NBA, NAAC - A+ Accredited)

Kanuru, Vijayawada - 520007



CERTIFICATE

This is to certify that the project work report titled “**STYLE SYNTHESIS: Bridging Art and Technology through Image Transfer**” is the bonafide work of **P.SRINITHI** (20501A1293), **M.SRIRAM** (20501A12B1) , **E.SANJANA** (20501A12C5) , **P.RISHITHA** (20501A1292) in partial fulfillment of the requirements for the award of the graduate degree of **BACHELOR OF TECHNOLOGY** in Information Technology during the academic year 2023-2024.

Signature of the Guide

**Mr.Ch.Praneeth.,
M.Tech,(Ph.D)**

Signature of the HOD

**Dr.B.V.SUBBA RAO.,
M.Tech., Ph.D., LMISTE., MIE**

ACKNOWLEDGEMENT

First and foremost we sincerely salute our esteemed institution **PRASAD V. POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY** for giving this golden opportunity for fulfilling our warm dreams of becoming engineers.

We hereby express our sincere gratitude to our principal **Dr. K. Sivaji Babu, M.Tech, Ph.D**, who has rendered his constant encouragement and valuable suggestions making my goals successful.

We are also thankful to our Head of Department **Dr. B. V. SubbaRao, M.Tech, Ph.D, LMISTE, MIE** for his constant encouragement and valuable support throughout the course of the project.

We are glad to express our deep sense of gratitude to **Mr.Ch.Praneeth, M.Tech, (Ph.D)**. Assistant professor, our guide, for his guidance and cooperation in completing this project successfully.

We thank one and all who have rendered help directly or indirectly in the Completion of this project successfully.

..... Project Associates

P.SRINITHI	(20501A1293)
M.SRIRAM	(20501A12B1)
E.SANJANA	(20501A12C5)
P.RISHITHA	(20501A1292)

ABSTRACT

The intersection of art and technology, showcased by deep learning models, offers innovative avenues for creative expression. As computational artistry progresses, refining style transfer techniques becomes pivotal for pushing artistic boundaries and enhancing visual content synthesis. Collaboration among artists, researchers, and technologists unlocks new potentials in computational creativity, reshaping artistic expression in the digital era. This paper comprehensively compares four prominent deep learning models—VGG19, VGG16, Magenta, and ResNet—in image style transfer. Through empirical evaluation using metrics like SSIM and PSNR, and qualitative assessment via human perceptual studies, their strengths and weaknesses are revealed. ResNet excels in preserving structural fidelity, VGG16 in maintaining fine details, and VGG19 in effectively applying style while maintaining quality. However, a holistic evaluation approach, considering both metrics and visual inspection, remains crucial for accurately assessing style transfer effectiveness amid ongoing research and refinement.

TABLE OF CONTENTS

Sl.No.	Description	Page No.
1. INTRODUCTION		1
1.1	PROBLEM DEFINITION	1
1.2	EXISTING SYSTEM	1
1.3	PROPOSED SYSTEM	2
1.4	SOFTWARE & HARDWARE REQUIREMENTS	3
1.5	MODULE DESCRIPTION	4
1.6	SYSTEM REQUIREMENT SPECIFICATION	5
2. SYSTEM ANALYSIS		7
2.1	FLOW ORIENTED MODELLING	7
2.1.1	DATA FLOW DIAGRAMS	7
2.1.2	FLOW CHARTS	13
3. DATA ANALYSIS		15
3.1	DATASET USED	15
3.2	TRAINING OF DATASET	17
4. SYSTEM IMPLEMENTATION		19
4.1	CONVOLUTIONAL NEURAL NETWORK	21
4.2	MODEL ARCHITECTURES	22
4.3	PYTHON	25
4.4	ANACONDA NAVIGATOR	26
5. SOURCE CODE		28
6. SCREENS		37
7. WORKING FUNCTIONALITY		38
8. FURTHER SCOPE FOR DEVELOPMENT		39
9. CONCLUSION		41
10. REFERENCES		42
11. APPENDIX -A		43

LIST OF FIGURES

FIGURE NO.	NAME OF FIGURES	PAGE NO.
1.	LEVEL-0 DFD	10
2.	LEVEL-1 DFD	11
3.	LEVEL-2 DFD	12
4.	FLOW CHART DIAGRAM	13
5.	CONTENT IMAGES DATASET	16
6.	STYLE IMAGES DATASET	16
7.	CONVOLUTIONAL NEURAL NETWORKS	19
8.	WORKING OF NST	21
9.	VGG16 ARCHITECTURE	23
10.	VGG19 ARCHITECTURE	24
11.	RESNET ARCHITECTURE	24

CHAPTER-1

INTRODUCTION

1.1 PROBLEM DEFINITION

Photographic style transfer involves the challenging task of seamlessly applying the aesthetic characteristics of a reference style image to various content images, maintaining photorealism and semantic constancy. Existing techniques, particularly those inspired by neural networks, encounter difficulties in achieving faithful transfers without introducing distortions reminiscent of paintings. The need arises for an innovative solution that can handle a wide spectrum of image content, accommodating variations in time of day, climate, season, and artistic edits, at the same time as preserving the intrinsic structure and semantic accuracy. The project aims to address these challenges by proposing a deep-learning approach that leverages locally affine constraints in color space, providing an effective and flexible approach to the problem of photorealistic style transfer.

1.2 EXISTING SYSTEM

The current landscape of photographic style transfer involves several existing methods, each with its own set of strengths and limitations. The prominent approaches include:

➤ **Neural Style Transfer:**

Neural Style Transfer techniques, such as those inspired by Gatys et al. [5], have been pivotal in the field. These methods leverage deep convolutional neural networks, like VGG-19, to separate content and style from images. However, one limitation is their susceptibility to producing distortions resembling paintings, even when applied to photographic inputs.

➤ **CNNMRF:**

CNNMRF (Convolutional Neural Network for Mutual Reinforcement Fields) introduced by Li et al. [10] utilizes a nearest-neighbor search in the feature space for style transfer. While effective in certain scenarios, it may generate partial style transfers, ignoring significant portions of the style image.

➤ **Global Style Transfer:**

Global style transfer algorithms, exemplified by Reinhard et al. [12] and Pitié et al. [11], apply spatially-invariant transfer functions to match color statistics globally. However, these methods are limited in handling spatially-varying color transformations and may lack flexibility in capturing nuanced stylistic changes.

1.2.1 DISADVANTAGES OF EXISTING SYSTEM

Despite their contributions, the existing style transfer methods exhibit certain drawbacks:

➤ **Neural Style Transfer:**

- Tends to produce distortions resembling paintings, limiting its suitability for photorealistic style transfer.
- May exhibit spillovers and ignore semantic context, impacting the faithfulness of style transfer.

➤ **CNNMRF:**

- Often generates partial style transfers, ignoring significant portions of the style image.
- Relies on nearest-neighbor search, potentially leading to mismatches and incomplete transfers.

➤ **Global Style Transfer:**

- Limited in handling spatially-varying color transformations, restricting its ability to capture nuanced stylistic changes.
- May lack flexibility in adapting to diverse content, resulting in less expressive style transfers.

These limitations highlight the need for an improved system that can overcome these challenges, aiming for photorealistic style transfer with enhanced fidelity, structural preservation, and semantic accuracy.

1.3 PROPOSED SYSTEM

The proposed system introduces a sophisticated method for transferring image styles, drawing on the latest advancements in deep learning technology. By combining cutting-edge models such as VGG19, VGG16, Magenta, and ResNet, it enables precise and high-fidelity stylization. Through the utilization of convolutional neural networks and customized optimization algorithms, the system strives to maintain the integrity of both content details and artistic styles. Its evaluation process includes thorough assessments using quantitative metrics like SSIM and PSNR, as well as qualitative human perceptual studies, ensuring both accuracy and perceptual quality in the stylized images produced. With a focus on adaptability and ease of use, the system offers real-time processing capabilities and a user-friendly interface designed for intuitive interaction. Ultimately, this system serves as a comprehensive

solution that merges the realms of art and technology, empowering users to unleash their creativity and express themselves artistically in the digital sphere.

1.3.1 ADVANTAGES OF PROPOSED SYSTEM

The proposed system offers several advantages, including:

- **Enhanced Creativity:** By integrating state-of-the-art deep learning models and optimization algorithms, the system empowers users to explore a wide range of artistic styles and express their creativity effectively.
- **Improved Accuracy:** Through comprehensive evaluation methodologies, encompassing both quantitative metrics and qualitative human perceptual studies, the system ensures accurate and aesthetically appealing stylized outcomes.
- **Real-time Processing:** Leveraging efficient algorithms and optimization techniques, the system enables real-time processing of style transfer tasks, allowing users to visualize and refine their creations instantly.
- **User-Friendly Interface:** With a user-friendly interface, the system provides a seamless and intuitive experience for users of all skill levels, facilitating easy exploration and application of artistic styles.
- **Adaptability:** The system is adaptable to diverse artistic styles and content types, allowing users to experiment with various visual elements and customize their stylized images according to their preferences.

1.4 SOFTWARE AND HARDWARE REQUIREMENTS

SOFTWARE REQUIREMENTS:

- A suitable operating system
- Deep Learning Framework: TensorFlow or PyTorch for neural network implementation.
- Programming Language: Python for coding the neural style transfer algorithm.
- Version Control: Git for version control and collaborative development.
- IDE (Integrated Development Environment): Jupyter Notebook or any preferred Python IDE for coding and experimentation.
- Libraries: NumPy, OpenCV, and other relevant Python libraries for image processing and manipulation.

HARDWARE REQUIREMENTS:

- CPU (Central Processing Unit): Multi-core processor to support overall system performance.
- RAM (Random Access Memory): 16GB or higher for handling large datasets and neural network operations.
- Storage: SSD (Solid State Drive) with sufficient capacity for storing datasets and model checkpoints.
- Display: High-resolution monitor for visualization and analysis during development.
- Internet Connection: Stable internet connection for accessing necessary libraries, frameworks, and collaborative tools.

1.5 MODULE DESCRIPTION

Step-1: Input Module: Accepts user input in the form of an image to be stylized.

Step-2: Reference Style Module: Allows the user to provide a reference style image that defines the desired artistic style for the output.

Step-3: Preprocessing Module: Prepares the input and reference images for neural style transfer. Handles tasks such as resizing, normalization, and data formatting.

Step-4: Neural Style Transfer Module: Utilizes a deep neural network to perform style transfer from the reference image to the input image. Implements the proposed locally affine transformation and custom differentiable energy term for photorealistic results.

Step-5: Semantic Segmentation Module: Incorporates semantic segmentation to enhance the transfer process by considering the context of different scene elements. Segments input and reference images to ensure style transfer occurs between semantically equivalent regions.

Step-6: Photorealism Regularization Module: Implements the photorealism regularization term to prevent distortions in the output. Ensures that the transformation applied to the input image is locally affine in color space.

Step-7: Output Module: Presents the final stylized image to the user. Provides options for user interaction, such as adjusting parameters or saving the result.

1.6 SYSTEM REQUIREMENT SPECIFICATION

The System Requirement Specification outlines the detailed specifications and constraints of the proposed project. It encompasses both functional and non-functional requirements essential for the development and deployment of the system. Functional requirements delineate the system's expected behavior, including features, modules, and interactions. Non-functional requirements encompass aspects like performance, security, and usability criteria. The SRS serves as a blueprint for the development team, guiding the design and implementation phases, and acts as a reference for stakeholders to validate whether the final product aligns with the specified criteria. It plays a crucial role in ensuring clarity, completeness, and coherence in the project's development lifecycle.

1.5.1 FUNCTIONAL REQUIREMENTS

In software engineering, a functional requirement defines a function of a software system or its component. A function is described as a set of inputs, behavior, and outputs (see also software). Functional requirements may be calculations, technical details, data manipulation and processing, and other specific functionality that define what a system is supposed to accomplish. In the Neural Style Transfer project, the functional requirements define the specific capabilities and features that the system must possess to achieve its objectives. These requirements encompass the core functionalities of the project, including the implementation of photorealism regularization, augmented style loss with semantic segmentation, and the integration of a two-stage optimization process. The system is expected to handle diverse scenarios such as time-of-day hallucination, weather changes, and artistic edits. Key functionalities include:

- Implementation of locally affine color transformations to preserve photorealism.
- Incorporation of semantic segmentation to enhance style transfer accuracy.
- Integration of the Matting Laplacian to constrain the transformation process.
- Utilization of the pre-trained VGG-19 model for feature extraction.
- Two-stage optimization using the Neural Style algorithm as an initial step.
- Compatibility with NVIDIA Titan X GPU for efficient processing.

These functional requirements serve as a detailed blueprint, guiding the development team towards the successful realization of the Neural Style Transfer project's key features and capabilities.

1.5.2 NON FUNCTIONAL REQUIREMENTS

In systems engineering and requirements engineering, a non-functional requirement is a requirement that specifies criteria that can be used to judge the operation of a system, rather than specific behaviors. The success of the Neural Style Transfer project relies not only on its specific functionalities but also on several non-functional aspects that contribute to its overall performance, usability, and reliability.

➤ **Performance:**

- The system should complete the style transfer process within a reasonable time frame, aiming for an average processing time of 3 to 5 minutes on an NVIDIA Titan X GPU. Response times for user interactions with the system.

➤ **Usability:**

- The user interface should be intuitive and user-friendly, allowing users to easily provide input images, reference style images, and semantic masks.
- The system should support a variety of input formats, including common image file types, to enhance usability and user convenience.

➤ **Reliability:**

- The system must consistently produce reliable and accurate style transfer results across diverse scenarios, ensuring faithful representation of the reference style.

➤ **Scalability:**

- The system should be scalable to accommodate potential future updates or enhancements, allowing for the integration of new features or improvements in style transfer algorithms.

➤ **Security:**

- The project should implement measures to ensure the security and privacy of user data, especially when dealing with input images and reference style images.

➤ **Compatibility:**

- The system should be compatible with standard operating systems, ensuring accessibility across different platforms such as Windows, macOS, and Linux.

These non-functional requirements provide a comprehensive framework for evaluating the overall effectiveness and reliability of the Neural Style Transfer project, ensuring it meets the expectations and demands of users.

CHAPTER-2

SYSTEM ANALYSIS

2.1 FLOW ORIENTED MODELING:

2.1.1 Data Flow Diagram (DFD):

A Data Flow Diagram (DFD) is also known as a Process Model. Process Modelling is an analysis technique used to capture the flow of inputs through a system (or group of processes) to their resulting output. The model is fairly simple in that there are only four types of symbols – process, data-flow, external entity, data store.

Process Modelling is used to visually represent what a system is doing. It is much easier to look at a picture and understand the essence than to read through verbiage describing the activities. System Analyst after talking with various users will create DFD diagrams and then show them to users to verify that their understanding is correct. The process models can be created to represent an existing system as well as a proposed system.

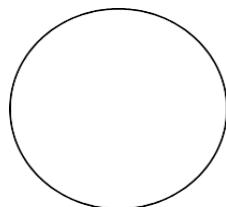
The following are some of the symbols in Process Modelling represents.

- Process
- Data Object
- Data Store
- External entity

Process:

An activity or a function that is performed for some specific reason; can be manual or Computerized; ultimately each process should perform only one activity.

- **Symbol:**



Data Flow:

Single piece of data or logical collection of information like an audio clip.

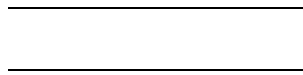
- **Symbol:**



Data Store:

Collection of data that is permanently stored.

- **Symbol:**

**External Entity:**

A person, organization, or system that is external to the system but intersects with it.

- **Symbol:**



The following are the levels of Data Flow Diagrams:

Level 0 DFD:

- The level 0 DFD (also known as the context level DFD) is the simplest DFD.
- The outermost level (Level 0) is concerned with how the system interacts with the outside world.
- This level basically represents the input and output of the entire system.

Level 1 DFD:

- The basic modules of the system are represented in this phase and how data moves through different module is shown.
- The level 1 DFD provides a high-level view of the system that identifies the major processes and data stores.

Level 2 DFD and other level of DFD:

- Each process from level 1 is exploded even more into processes. This decomposition continues for each level.
- The number of levels possible depends on the complexity of the system.

DFD Principles

- The general principle in Data Flow Diagramming is that a system can be decomposed into subsystems, and subsystems can be decomposed into lower-level subsystems, and so on.
- Each subsystem represents a process or activity in which data is processed. At the lowest level, processes can no longer be decomposed.
- Each 'process' (and from now on, by 'process' we mean subsystem and activity) in a DFD has the characteristics of a system.
- Just as a system must have input and output (if it is not dead), so a process must have input and output.
- Data enters the system from the environment; data flows between processes within the system; and data is produced as output from the system.

Salient Features of DFD'S

- The DFD shows flow of data, not of control loops and decisions are controlled considerations that do not appear on a DFD.
- The DFD does not indicate the time factor involved in any process whether the data flow takes place daily, weekly, monthly or yearly.
- The sequence of events is not brought out on the DFD.

Constructing a DFD:

Several rules of thumb are used in drawing DFD'S:

- Processes should be named and numbered for an easy reference. Each name should be representative of the process.
- The direction of flow is from top to bottom and from left to right. Data traditionally flows from source to destination although they may flow back to the source. One way to indicate this is to draw a long flow line back to a source.
- An alternative way is to repeat the source symbol as a destination. Since it is used more than once in the DFD it is marked with a short diagonal.

- An alternative way is to repeat the source symbol as a destination. Since it is used more than once in the DFD it is marked with a short diagonal.
- The names of data stores and destinations are written in capital letters. Process and data-flow names have the first letter of each word capitalized.
- A DFD typically shows the minimum contents of a data store. Each data store should contain all the data elements that flow in and out.

Level-0 DFD:



Fig 1: Level-0 DFD

DESCRIPTION:

The Level 0 Data Flow Diagram (DFD) illustrates the high-level view of a Neural Style Transfer System. It showcases the key entities and their interactions within the system. The actors involved are the User and the Content Provider. The process involves loading both the Style and Content Images, which are then used in the style transfer operation. The output, consisting of the stylized image, is presented to the User, who has the option to either display it or save it. This diagram provides a concise overview of the primary components and the flow of data within the Neural Style Transfer System, laying the foundation for a more detailed exploration in subsequent DFD levels.

Level-1 DFD:

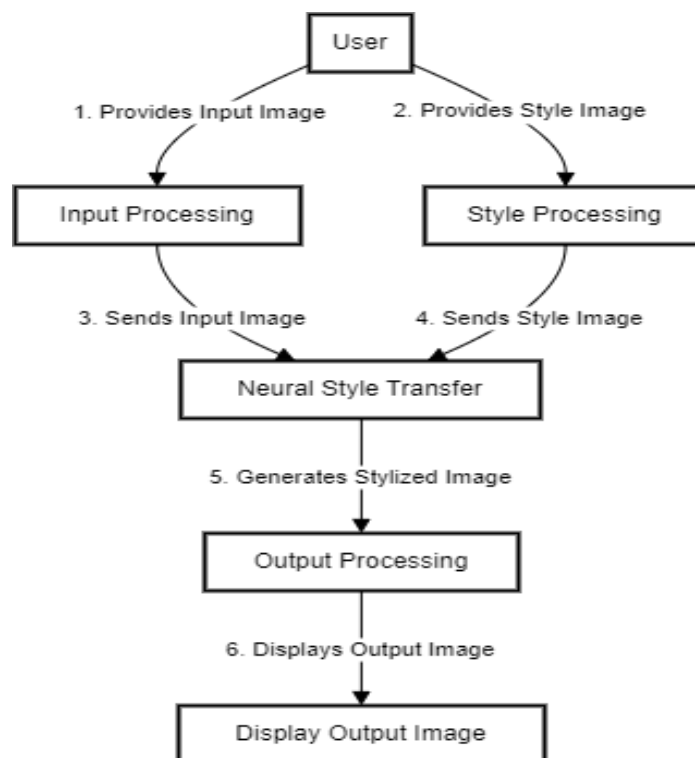


Fig 2: Level-1 DFD

DESCRIPTION:

The Level 1 Data Flow Diagram (DFD) for the Neural Style Transfer System delves into more detail, breaking down the processes outlined in the Level 0 diagram. It introduces specific actions and components involved in each stage. The User and Content Provider, now identified as distinct entities, initiate the process by loading Style and Content Images individually. These images undergo preprocessing, including steps such as resizing and normalization. The preprocessed images are then used to construct the Neural Network. The system performs style transfer by employing this constructed Neural Network, leading to the generation of a stylized output. The User can choose to either display or save the output image. This detailed diagram provides a comprehensive view of the system's inner workings, highlighting the sequential steps involved in Neural Style Transfer, from image loading to the final output.

Level-2 DFD:

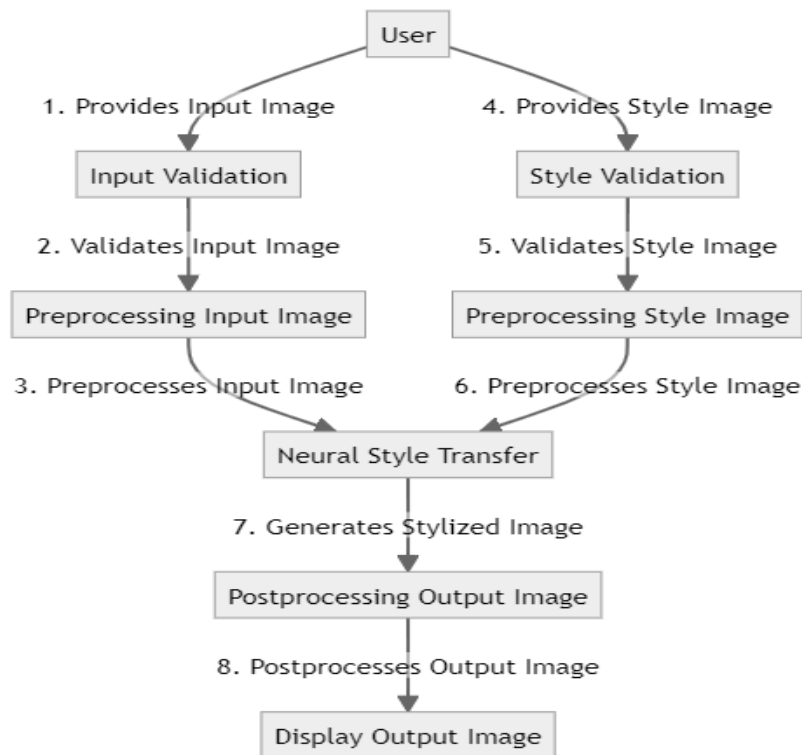


Fig 3: Level-2 DFD

DESCRIPTION:

The Level 2 Data Flow Diagram (DFD) for the Neural Style Transfer System provides a more detailed view of the sub processes identified in the Level 1 diagram. Each sub process is further decomposed to illustrate the specific activities and data flows within. "Image Loading" now involves the User selecting Style and Content Images, which are then loaded and processed separately. The "Preprocessing" stage breaks down into resizing and normalizing Style and Content Images individually. "Neural Network Construction" delves into the extraction of style and content features, the construction of Gram Matrices, and the subsequent building of the Neural Network, each step represented separately. The "Style Transfer" phase emphasizes the forward pass with the Content Image, leading to the generation of a stylized image. Lastly, the "Output & Storage" section elaborates on displaying and saving the stylized image for the User. The Level 2 DFD offers a granular depiction of the intricate processes within each major stage of the Neural Style Transfer System, enhancing understanding and clarity.

2.1.2 Flow Chart Diagram:

A flowchart is a type of diagram that represents an algorithm or process, showing the steps as boxes of various kinds, and their order by connecting them with arrows. This diagrammatic representation illustrates a solution to a given problem. Process operations are represented in these boxes, and arrows; rather, they are implied by the sequencing of operations. Flowcharts are used in analyzing, designing, documenting or managing a process or program in various fields. A flow chart diagram typically consists of shapes such as ovals, diamonds and boxes that contain words relating to a certain issue and lines with arrows connecting boxes to one another. Flowcharts are used in designing and documenting complex processes or programs. Like other types of diagrams, they help visualize what is going on and thereby help the viewer to understand a process, and perhaps also find flaws, bottlenecks, and other less-obvious features within it. There are many different types of flowcharts, and each type has its own repertoire of boxes and notational conventions. The two most common types of boxes in a flowchart are:

- A processing step, usually called activity, and denoted as a rectangular box.
- A decision usually denoted as a diamond.

Flow chart diagram:

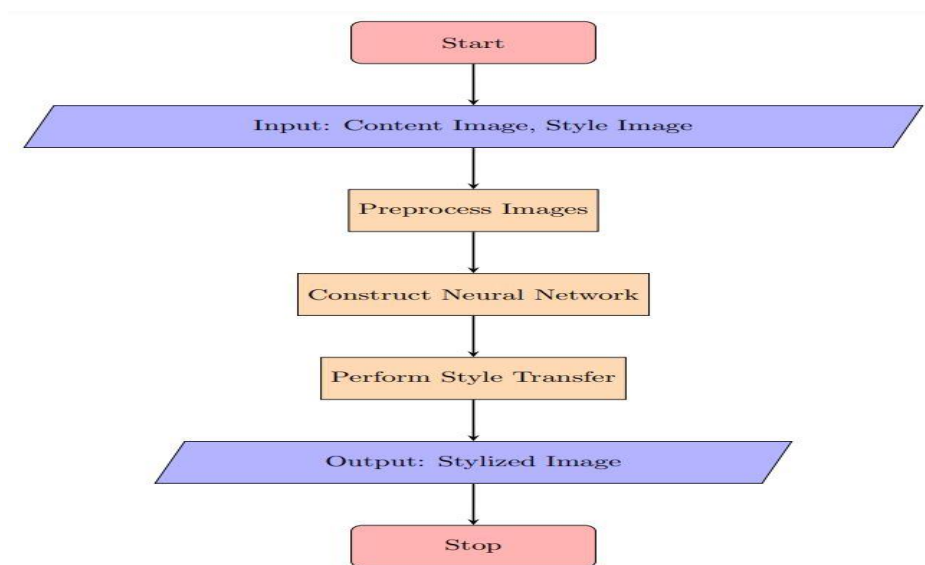


Fig 3: Flow Chart Diagram

DESCRIPTION:

The flowchart outlines the step-by-step execution of the neural style transfer algorithm. It commences with the "Start" node, denoting the initiation of the process. The required input, comprising a content image and a style image, is then received. Moving to the "Preprocess Images" stage, the input images undergo necessary transformations, such as resizing or normalization, to prepare them for subsequent processing.

The algorithm's core, the construction of a neural network, is highlighted in the "Construct Neural Network" block. This step involves extracting both style and content features from the preprocessed images, contributing to the network's ability to understand and recreate the desired artistic style. The subsequent "Perform Style Transfer" block signifies the application of the trained neural network to generate a stylized version of the content image based on the style image.

The final stages involve presenting the outcome through the "Output: Stylized Image" block and concluding the process with the "Stop" node. Overall, this comprehensive flowchart provides a visual roadmap of the neural style transfer procedure, aiding in understanding the intricacies involved in transforming a given content image to adopt the artistic style of another image.

CHAPTER-3

DATA ANALYSIS

3.1 Dataset Used

The dataset utilized in this project plays a pivotal role in evaluating and comparing the performance of deep learning models for image style transfer. Comprising pairs of content and style images, the dataset serves as the foundation for training and evaluating the style transfer models. It consists of high-resolution images sourced from diverse artistic styles, genres, and movements, reflecting the rich diversity of visual artistry.

Each pair in the dataset consists of a content image, depicting the object or scene intended for style transformation, and a style image representing the artistic style to be applied. These images are carefully curated to encompass a wide range of visual characteristics, including textures, color palettes, brush strokes, and compositions, essential for comprehensive model evaluation. The diversity within the dataset ensures that the style transfer models are tested across various artistic styles, enabling a robust comparative analysis.

The content images within the dataset span different content types, ranging from natural landscapes and architectural scenes to portraits and still-life compositions. This diversity in content allows for a thorough assessment of the models' flexibility and adaptability in handling different subject matters and scene complexities. Furthermore, the inclusion of high-resolution images enhances the fidelity and detail preservation aspect of the style transfer evaluation, ensuring a more nuanced understanding of model performance.

Overall, the dataset used in this project serves as a comprehensive benchmark for evaluating the effectiveness of deep learning models in image style transfer. Its diverse collection of content and style images enables researchers to assess various aspects of model performance, including fidelity, computational efficiency, and adaptability to different artistic styles. By leveraging this rich dataset, the project aims to provide valuable insights into the relative strengths and weaknesses of the evaluated models, contributing to the advancement of computational artistry and creative expression.

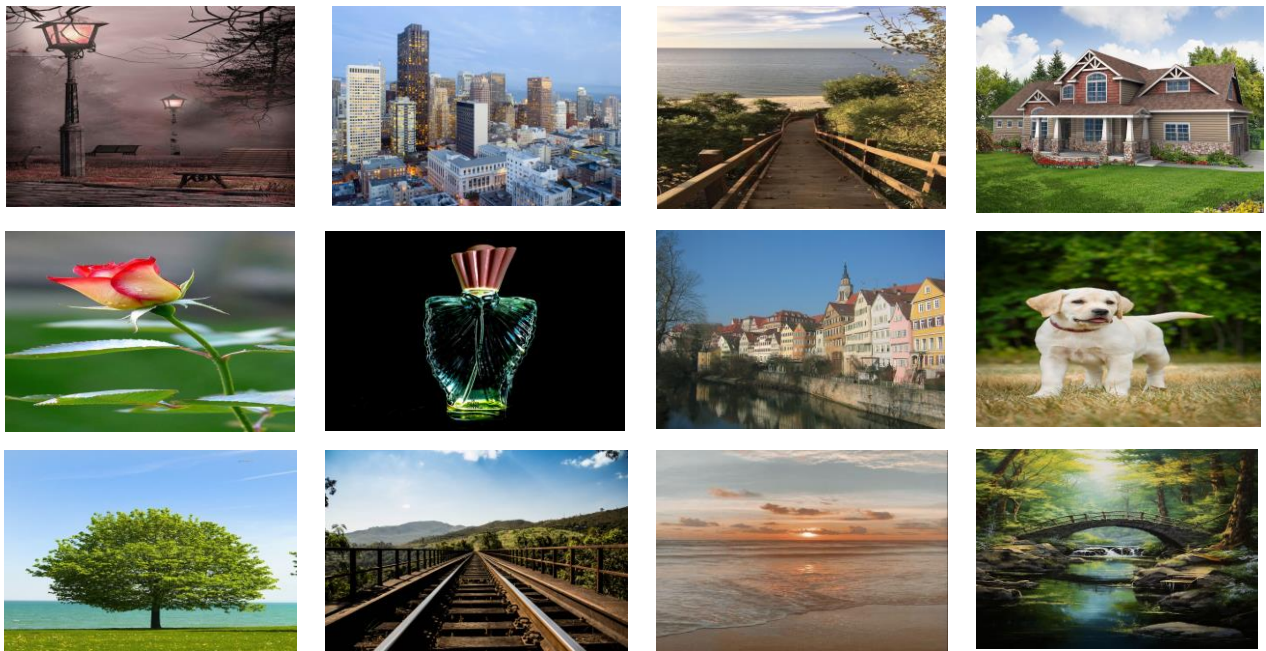
DATASET:

Fig 4: Content Images dataset

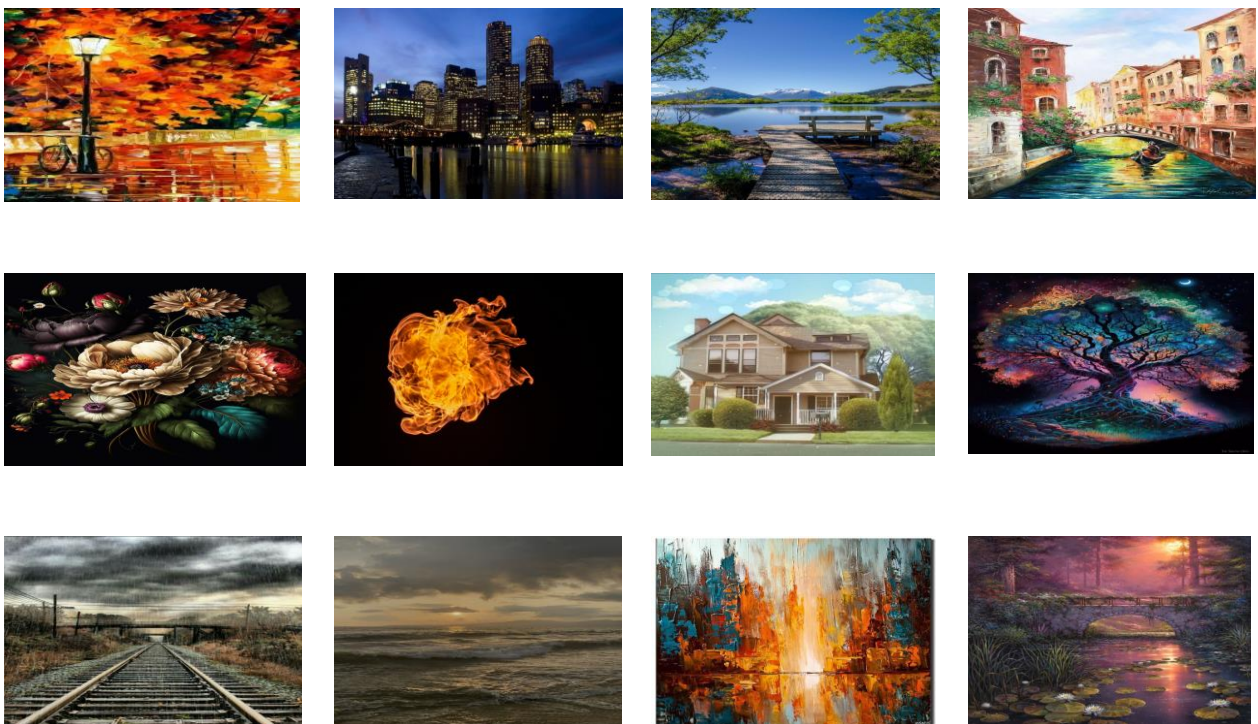


Fig 5: Style Images dataset

3.2 Training of Dataset

In the context of this project, the training dataset serves as the cornerstone for evaluating and refining deep learning models in the domain of image style transfer. Comprising meticulously paired images, the dataset encapsulates both content and style representations essential for the style transfer process. Each pair consists of a content image, delineating the subject matter intended for stylization, and a corresponding style image, encapsulating the artistic essence to be emulated.

Composition of the Training Dataset:

The composition of the training dataset is carefully curated to encapsulate a diverse array of artistic styles, genres, and visual motifs. This diversity is crucial for ensuring that the trained models can effectively adapt to a wide spectrum of aesthetic nuances, ranging from the subtle brushstrokes of impressionism to the bold geometric forms of cubism. By incorporating a rich tapestry of stylistic elements, the dataset empowers the models to discern and replicate the intricate nuances of various artistic expressions.

Diversity and Representation:

One of the primary objectives in crafting the training dataset is to ensure comprehensive coverage of artistic styles and visual characteristics. From classical masterpieces to contemporary avant-garde creations, the dataset spans across epochs and movements, offering a holistic panorama of artistic diversity. Moreover, the dataset encompasses a spectrum of content types, including landscapes, portraits, still lifes, and abstract compositions, further enriching the models' learning experience.

Quality and Resolution:

The quality and resolution of the images within the training dataset are paramount considerations to uphold fidelity and detail preservation during the style transfer process. High-resolution images are preferred to capture intricate details and subtle nuances inherent in both content and style representations. By prioritizing image quality, the dataset aims to imbue the trained models with a keen sensitivity to the finer aspects of artistic expression, thereby enhancing the authenticity of stylized outputs.

Dataset Size and Scalability:

The scalability of the training dataset is pivotal in accommodating the learning requirements of deep learning models. A sizable dataset facilitates robust learning across a myriad of stylistic variations and content compositions, enabling the models to generalize effectively to unseen data. Moreover, as research advances and new stylistic paradigms emerge, the scalability of the dataset ensures its adaptability to evolving trends and artistic sensibilities, thereby fostering continual innovation in computational artistry.

In essence, the training dataset serves as the bedrock of the data analysis process, underpinning the evaluation and refinement of deep learning models for image style transfer. Its rich composition, encompassing diverse artistic styles, genres, and visual motifs, fosters a comprehensive understanding of model performance and efficacy. Through meticulous curation and attention to detail, the dataset empowers the models to traverse the nuanced landscape of artistic expression, ultimately reshaping the boundaries of computational creativity in the digital age.

CHAPTER-4

SYSTEM IMPLEMENTATION

4.1 Convolutional Neural Network

A Convolutional Neural Network (CNN) is a deep learning model tailored for structured grid data like images. It draws inspiration from the organization of the animal visual cortex and comprises several layers: convolutional, pooling, and fully connected. Convolutional layers employ filters to extract features through convolutions, while pooling layers reduce spatial dimensions for computational efficiency and translation invariance. Fully connected layers aggregate these features to make predictions based on learned patterns. CNNs have excelled in diverse computer vision tasks, including image classification, object detection, and style transfer, thanks to their capacity to automatically learn hierarchical representations of visual data.

CNNs' hierarchical and compositional nature enables them to capture intricate patterns and relationships in images. This characteristic makes CNNs indispensable tools in modern image processing and pattern recognition applications. By automatically learning features at different levels of abstraction, CNNs excel in discerning complex patterns, contributing significantly to advancements in fields such as autonomous driving, medical imaging, and natural language processing. As a cornerstone of deep learning, CNNs continue to evolve with novel architectures and training techniques, further enhancing their capability to extract meaningful insights from visual data.

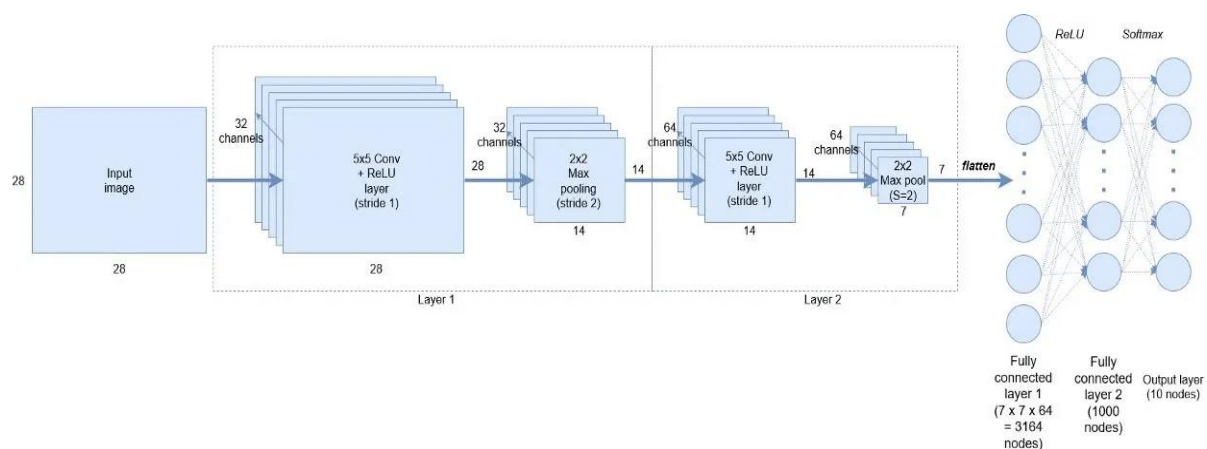


Fig 6: Convolutional Neural Network

In the initial layer of a Convolutional Neural Network (CNN), with 32 filters, the network identifies basic patterns like straight or horizontal lines, which may seem insignificant to us but hold great importance for the network. As we progress to Layer 2, equipped with 64 filters, the network begins to recognize increasingly complex features such as a dog's face or a car's wheel. This process of detecting various simple and intricate features is termed feature representation.

It's crucial to understand that CNNs don't comprehend the content of images; instead, they encode what each image represents. This inherent encoding capability of Convolutional Neural Networks is particularly beneficial in tasks like Neural Style Transfer. Delving deeper into this concept sheds more light on its functionality.

How Convolutional Neural Networks are used to capture Content and Style of images?

Convolutional Neural Networks (CNNs) play a crucial role in capturing both the content and style of images through their hierarchical architecture. At the initial layers, CNNs detect simple features like edges and textures, gradually progressing to more complex structures such as objects and faces in deeper layers. This hierarchical feature extraction enables CNNs to encode the essential elements present in an image's content. Additionally, CNNs analyze textures, colors, and patterns to capture the unique stylistic characteristics of images. Through training on diverse datasets, CNNs learn to identify and extract these stylistic elements, allowing them to encode the artistic style of an image effectively.

Utilizing the encoding nature of CNNs, Neural Style Transfer algorithms can blend the content of one image with the style of another. By optimizing the generated image to match the content features of a target content image and the style features of a reference style image, CNNs facilitate the synthesis of visually appealing images that seamlessly combine content and style. This process demonstrates the remarkable ability of CNNs to capture and reproduce both the content and style aspects of images, making them invaluable tools for various image analysis and synthesis tasks.

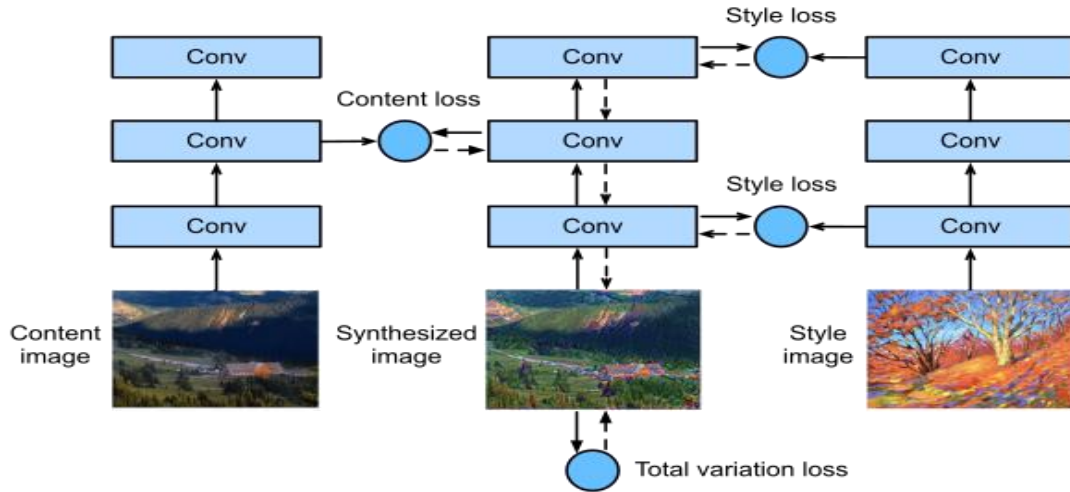


Fig 7: Working of NST

Content loss, style loss, Gram matrix, and the total loss function are fundamental components of Neural Style Transfer (NST) algorithms, leveraging Convolutional Neural Networks (CNNs) to blend the content and style of images effectively.

In Neural Style Transfer, content loss measures how much the generated image deviates from the content of the target image, while style loss quantifies the difference in stylistic features between the generated image and the style reference image. The Gram matrix is a crucial component used to compute style loss. The total loss function combines content and style losses to optimize the generated image.

Content Loss: Content loss measures the similarity between the features of the generated image (G) and the target content image (C). It is typically computed using a distance metric such as Mean Squared Error (MSE) between the feature representations of the generated image and the content image at a certain layer of a pre-trained CNN:

$$L_{\text{content}}(C, G) = (1/2) \sum_{i,j} (F_{ij}^C - F_{ij}^G)^2$$

Where F_{ij}^C and F_{ij}^G represent the feature representations of the content and generated images at layer i and position j in the feature maps.

Gram Matrix: The Gram matrix is computed from the feature maps of a given layer in the CNN. It represents the correlation between different features and encodes the style information of an image:

$$G_{ij}^l = \sum_k F_{ik}^l F_{jk}^l$$

Where G_{ij}^l is the element at position (i,j) in the Gram matrix for layer l, and F_{ik}^l and F_{jk}^l are the activations of the k-th feature map at layer l at positions i and j, respectively.

Style Loss: Style loss captures the difference in style between the generated image (G) and the style reference image (S). It is computed by comparing the Gram matrices of the feature maps of the generated image and the style reference image at multiple layers of the CNN:

$$L_{\text{style}}(S, G) = \sum_{l=0}^L w_l E_l$$

Where L is the total number of layers considered, E_l is the style loss at layer l, and w_l is a weighting factor for each layer.

Total Loss Function: The total loss function combines the content and style losses using weighting factors α and β to balance their contributions:

$$L_{\text{total}}(C, S, G) = \alpha L_{\text{content}}(C, G) + \beta L_{\text{style}}(S, G)$$

where α and β control the relative importance of content and style in the generated image. Adjusting these parameters allows fine-tuning of the final output to achieve the desired balance between content fidelity and stylistic resemblance.

4.2 Models Architectures

4.2.1 Magenta:

The TensorFlow Hub model "magenta/arbitrary-image-stylization-v1-256/2" offers a pre-trained solution for arbitrary image stylization, enabling the application of diverse artistic styles to images. Although the intricate architecture of this model may not be readily accessible, a general overview can be provided. It operates on a convolutional neural network (CNN) framework trained to transfer the style from a given reference image to a content

image while retaining the content of the original image. The CNN architecture likely encompasses various layers such as convolutional layers, normalization layers (e.g., batch normalization), activation functions (e.g., ReLU), and potentially pooling layers. However, specific details regarding the architecture might not be publicly disclosed or readily available.

4.2.2 VGG 16:

The VGG-16 architecture is a deep convolutional neural network (CNN) model renowned for its simplicity and effectiveness in image classification tasks. It comprises 13 convolutional layers organized into five blocks, each followed by a max-pooling layer, with Rectified Linear Unit (ReLU) activation functions after each convolutional operation. Three fully connected layers follow the convolutional blocks, culminating in a softmax layer for class probability prediction. Characterized by its uniformity and use of 3x3 convolutional filters with a stride of 1, VGG-16 achieved state-of-the-art performance on various benchmarks upon its introduction. Despite its computational demands due to increased depth, VGG-16 remains widely utilized in computer vision applications, particularly for tasks such as feature extraction and transfer learning.

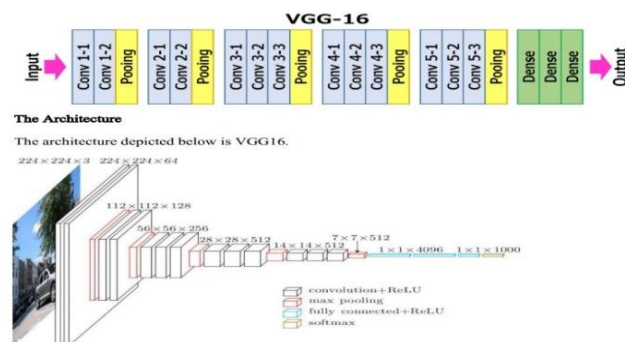


Figure 8: VGG16 architecture

4.2.3 VGG 19:

The VGG-19 architecture is an extension of the VGG-16 model and is recognized for its deeper network structure. Like VGG-16, VGG-19 consists of 19 layers, organized into convolutional blocks followed by max-pooling layers. The main difference lies in its increased depth, featuring additional convolutional layers within each block. This augmentation enhances the model's capacity to capture complex features from input images, potentially leading to improved performance on various computer vision tasks. Like VGG-16, VGG-19 employs 3x3 convolutional filters with ReLU activation functions, maintaining uniformity throughout the network architecture. Despite its deeper architecture, VGG-19 shares the same computational demands as VGG-16, which can be considerable. Nonetheless,

VGG-19 remains a popular choice in the field of deep learning, particularly when a higher level of feature representation is required, such as in image recognition and classification tasks.

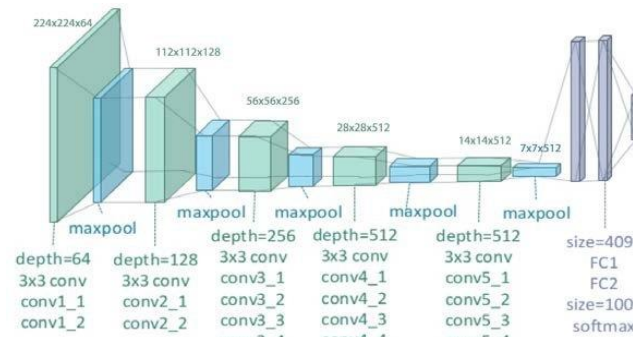


Figure 9: VGG19 architecture

4.2.4 ResNet:

The ResNet architecture, short for Residual Networks, introduced a groundbreaking innovation in deep learning by training very deep neural networks. ResNet's key contribution lies in the concept of residual learning, which involves the use of skip connections or shortcuts to jump over some layers in a neural network. These shortcuts facilitate the flow of information, allowing gradients to propagate more effectively during training, thereby mitigating the vanishing gradient problem. This enables reaching depths of over a hundred layers while maintaining or even improving performance. The architecture typically consists of residual blocks, where each block contains several convolutional layers with identity mappings or shortcut connections. ResNet architectures, such as ResNet-50 and ResNet-101, have demonstrated superior performance on various computer vision tasks, including image classification, object detection, and semantic segmentation, making them a cornerstone in modern deep learning research and applications.

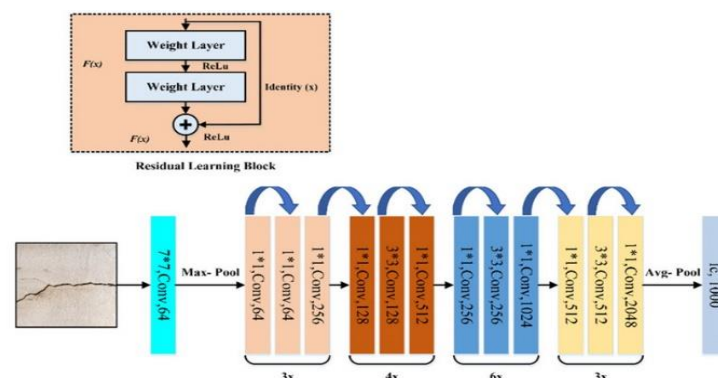


Figure 10: ResNet architecture

Structural Similarity Index (SSIM) and Peak Signal-to-Noise Ratio (PSNR) are both metrics commonly used to evaluate the quality of images.

Structural Similarity Index (SSIM): SSIM measures the similarity between two images by comparing their structure, luminance, and contrast. It calculates a value between -1 and 1, where 1 indicates perfect similarity. SSIM considers the perceived changes in structural information, making it more robust to variations in image content and illumination.

Peak Signal-to-Noise Ratio (PSNR): PSNR, on the other hand, quantifies the quality of an image by measuring the ratio between the maximum possible power of a signal and the power of corrupting noise. It's expressed in decibels (dB), and a higher PSNR value indicates better image quality. PSNR is particularly sensitive to changes in pixel values and is widely used in applications where preserving fine details is essential.

Both SSIM and PSNR provide valuable insights into image quality, but they have different strengths and weaknesses. SSIM is more aligned with human perception and is suitable for assessing overall image similarity, while PSNR is more straightforward and suitable for scenarios where preserving pixel values accurately is crucial.

4.3 Python

Python, a versatile and widely-used programming language, played a pivotal role in the development of the Neural Style Transfer system. Known for its simplicity, readability, and extensive library support, Python provided the ideal environment for implementing complex algorithms and handling large datasets efficiently. Leveraging libraries such as TensorFlow and PyTorch, Python facilitated the creation and training of deep neural networks, enabling seamless experimentation and rapid prototyping. Additionally, Python's rich ecosystem of scientific computing libraries, including NumPy, SciPy, and Matplotlib, offered robust tools for data manipulation, numerical computation, and visualization, enhancing the analysis and interpretation of results. Its open-source nature and active community further fostered collaboration and knowledge sharing, empowering developers to build sophisticated machine learning systems like Neural Style Transfer with ease. In summary, Python served as the backbone of the project, offering a powerful and flexible platform for implementing innovative solutions in the realm of artificial intelligence and image processing.

4.4 Anaconda Navigator

Anaconda Navigator provided a user-friendly interface for managing Python packages, environments, and applications. It facilitated seamless package installation and environment setup, ensuring compatibility and reproducibility across different systems. With Anaconda Navigator, developers could easily create isolated Python environments tailored to the specific requirements of their project, thereby avoiding dependency conflicts and ensuring consistent behavior across different development stages.

4.4.1 Jupyter Notebook:

Jupyter Notebook, a web-based interactive computing environment, offered a versatile platform for prototyping, experimentation, and documentation. Its support for literate programming enabled developers to combine code, visualizations, and explanatory text in a single interactive document, fostering clear communication and streamlined workflow. Through Jupyter Notebook's intuitive interface, developers could execute code cells interactively, visualize intermediate results, and iteratively refine their algorithms, thereby accelerating the development cycle and facilitating rapid experimentation.

One of the key advantages of using Anaconda Navigator and Jupyter Notebook was their seamless integration with popular Python libraries and frameworks such as TensorFlow, PyTorch, and Matplotlib. Developers could leverage these libraries within the Jupyter environment to implement complex machine learning algorithms, visualize data, and analyze results in real-time. Furthermore, Jupyter Notebook's support for rich media output, including images and interactive plots, enhanced the presentation of results and facilitated deeper insights into the Neural Style Transfer process.

4.4.2 NumPy and Matplotlib:

NumPy and Matplotlib, fundamental libraries for numerical computing and data visualization respectively, played crucial roles in the development process. NumPy provided essential support for numerical operations and array manipulation, facilitating efficient data processing and manipulation. Matplotlib enabled developers to create insightful visualizations, including plots, charts, and histograms, thereby enhancing the analysis and interpretation of experimental results.

4.4.3 TensorFlow:

TensorFlow, a powerful open-source machine learning framework developed by Google, served as the backbone of the Neural Style Transfer system. With its comprehensive set of tools and libraries for building and training deep neural networks, TensorFlow enabled developers to implement complex neural architectures, optimize model performance, and deploy models at scale. Leveraging TensorFlow's high-level APIs and pre-trained models, developers could expedite the development process and focus on fine-tuning the Neural Style Transfer algorithm to achieve optimal results.

4.4.4 Pillow (PIL):

Pillow, also known as the Python Imaging Library (PIL), provided essential support for image processing and manipulation within the Neural Style Transfer system. With its comprehensive suite of functions and utilities for image loading, transformation, and enhancement, Pillow enabled developers to preprocess input images, extract features, and generate stylized outputs with precision and efficiency. By integrating Pillow into the development pipeline, developers could seamlessly handle various image formats, apply artistic filters, and enhance the visual quality of the generated images.

4.4.5 IPython:

IPython, an interactive Python shell, offered enhanced interactivity and productivity features for the development and experimentation phase. Its support for inline plotting, tab completion, and rich media display enhanced the coding experience within Jupyter Notebook, enabling developers to explore data, experiment with algorithms, and iterate on design choices in real-time. By leveraging IPython's interactive capabilities, developers could streamline the development process, troubleshoot issues more effectively, and gain deeper insights into the behavior of the Neural Style Transfer algorithm.

In summary, the seamless integration of Anaconda Navigator, Jupyter Notebook, NumPy, Matplotlib, TensorFlow, Pillow, and IPython provided a robust and versatile development environment for the implementation of the Neural Style Transfer system. These tools and libraries collectively empowered developers to leverage the power of Python for image processing, deep learning, and visualization, thereby facilitating the creation of compelling and visually striking artistic renderings.

CHAPTER 5

SOURCE CODE

Code for Arbitrary Stylization Model (magenta) and VGG19

#Importing libraries

```
import os
import tensorflow as tf
import numpy as np
import PIL.Image
import time
import functools
```

Load compressed models from tensorflow_hub

```
os.environ['TFHUB_MODEL_LOAD_FORMAT'] = 'COMPRESSED'
import IPython.display as display
import tensorflow as tf
import matplotlib.pyplot as plt
import matplotlib as mpl
mpl.rcParams['figure.figsize'] = (12, 12)
mpl.rcParams['axes.grid'] = False
```

#tensor to image function

```
def tensor_to_image(tensor):
    tensor = tensor*255
    tensor = np.array(tensor, dtype=np.uint8)
    if np.ndim(tensor)>3:
        assert tensor.shape[0] == 1
        tensor = tensor[0]
    return PIL.Image.fromarray(tensor)
```

#accessing content and style paths

```
content_path = './img/inputs/in10.png'
style_path = './img/styles/st10.png'
```

#loading image from its path

```
def load_img(path_to_img):
    max_dim = 512
```

```

img = tf.io.read_file(path_to_img)
img = tf.image.decode_image(img, channels=3)
img = tf.image.convert_image_dtype(img, tf.float32)
shape = tf.cast(tf.shape(img)[: -1], tf.float32)
long_dim = max(shape)
scale = max_dim / long_dim
new_shape = tf.cast(shape * scale, tf.int32)
img = tf.image.resize(img, new_shape)
img = img[tf.newaxis, :]
return img

```

Function to display image

```

def imshow(image, title=None):
    if len(image.shape) > 3:
        image = tf.squeeze(image, axis=0)
    plt.imshow(image)
    if title:
        plt.title(title)

```

#plotting content and style images to screen

```

content_image = load_img(content_path)
style_image = load_img(style_path)
plt.subplot(1, 2, 1)
imshow(content_image, 'Content Image')
plt.subplot(1, 2, 2)
imshow(style_image, 'Style Image')

```

#arbitrary image stylization magenta model

```

import tensorflow_hub as hub
hub_model = hub.load('https://tfhub.dev/google/magenta/arbitrary-image-stylization-v1-256/2')
stylized_image = hub_model(tf.constant(content_image), tf.constant(style_image))[0]
tensor_to_image(stylized_image)

```

Resize the stylized image to match the dimensions of the content image

```

stylized_image_resized = tf.image.resize(stylized_image, tf.shape(content_image)[1:3])

```

Compute quantitative metrics for the stylized image

```

ssim_hub = tf.image.ssim(stylized_image_resized, content_image, max_val=1.0)

```

```
psnr_hub = tf.image.psnr(stylized_image_resized, content_image, max_val=1.0)
print("Arbitrary Image Stylization Network SSIM:", ssim_hub.numpy())
print("Arbitrary Image Stylization Network PSNR:", psnr_hub.numpy())
```

VGG19 Model

#preprocessing the images

```
x = tf.keras.applications.vgg19.preprocess_input(content_image*255)
x = tf.image.resize(x, (224, 224))
vgg = tf.keras.applications.VGG19(include_top=True, weights='imagenet')
prediction_probabilities = vgg(x)
prediction_probabilities.shape
```

#returning top 5 predicted classes

```
predicted_top_5 =
tf.keras.applications.vgg19.decode_predictions(prediction_probabilities.numpy())[0]
[(class_name, prob) for (number, class_name, prob) in predicted_top_5]
```

#Extracting all the layers

```
vgg = tf.keras.applications.VGG19(include_top=False, weights='imagenet')
print()
for layer in vgg.layers:
    print(layer.name)
```

#selecting the content and style layers

```
content_layers = ['block5_conv2']
style_layers = ['block1_conv1',
                'block2_conv1',
                'block3_conv1',
                'block4_conv1',
                'block5_conv1']
```

```
num_content_layers = len(content_layers)
```

```
num_style_layers = len(style_layers)
```

Loading pretrained VGG, trained on ImageNet data

```
def vgg_layers(layer_names):
    """ Creates a VGG19 model that returns a list of intermediate output values. """
    vgg = tf.keras.applications.VGG19(include_top=False, weights='imagenet')
    vgg.trainable = False
```

```

outputs = [vgg.get_layer(name).output for name in layer_names]
model = tf.keras.Model([vgg.input], outputs)
return model

```

```

style_extractor = vgg_layers(style_layers)
style_outputs = style_extractor(style_image*255)
# Look at the statistics of each layer's output
for name, output in zip(style_layers, style_outputs):
    print(name)
    print(" shape: ", output.numpy().shape)
    print(" min: ", output.numpy().min())
    print(" max: ", output.numpy().max())
    print(" mean: ", output.numpy().mean())
    print()

```

#defining gram matrix

```

def gram_matrix(input_tensor):
    result = tf.linalg.einsum('bijc,bijd->bcd', input_tensor, input_tensor)
    input_shape = tf.shape(input_tensor)
    num_locations = tf.cast(input_shape[1]*input_shape[2], tf.float32)
    return result/(num_locations)

```

#defining style and content model function

```

class StyleContentModel(tf.keras.models.Model):
    def __init__(self, style_layers, content_layers):
        super(StyleContentModel, self).__init__()
        self.vgg = vgg_layers(style_layers + content_layers)
        self.style_layers = style_layers
        self.content_layers = content_layers
        self.num_style_layers = len(style_layers)
        self.vgg.trainable = False
    def call(self, inputs):
        "Expects float input in [0,1]"
        inputs = inputs*255.0
        preprocessed_input = tf.keras.applications.vgg19.preprocess_input(inputs)
        outputs = self.vgg(preprocessed_input)

```

```

style_outputs, content_outputs = (outputs[:self.num_style_layers],
                                   outputs[self.num_style_layers:])
style_outputs = [gram_matrix(style_output)
                 for style_output in style_outputs]
content_dict = {content_name: value
               for content_name, value
               in zip(self.content_layers, content_outputs)}
style_dict = {style_name: value
             for style_name, value
             in zip(self.style_layers, style_outputs)}
return {'content': content_dict, 'style': style_dict}

```

```

extractor = StyleContentModel(style_layers, content_layers)
results = extractor(tf.constant(content_image))
print('Styles:')
for name, output in sorted(results['style'].items()):
    print(" ", name)
    print("  shape: ", output.numpy().shape)
    print("  min: ", output.numpy().min())
    print("  max: ", output.numpy().max())
    print("  mean: ", output.numpy().mean())
    print()
print("Contents:")
for name, output in sorted(results['content'].items()):
    print(" ", name)
    print("  shape: ", output.numpy().shape)
    print("  min: ", output.numpy().min())
    print("  max: ", output.numpy().max())
    print("  mean: ", output.numpy().mean())

```

```

style_targets = extractor(style_image)['style']
content_targets = extractor(content_image)['content']
image = tf.Variable(content_image)

```

```

def clip_0_1(image):
    return tf.clip_by_value(image, clip_value_min=0.0, clip_value_max=1.0)
opt = tf.keras.optimizers.Adam(learning_rate=0.02, beta_1=0.99, epsilon=1e-1)
style_weight = 1e-2
content_weight = 1e4
#calculating content and style loss
def style_content_loss(outputs):
    style_outputs = outputs['style']
    content_outputs = outputs['content']
    style_loss = tf.add_n([tf.reduce_mean((style_outputs[name]-style_targets[name])**2)
                           for name in style_outputs.keys()])
    style_loss *= style_weight / num_style_layers

    content_loss = tf.add_n([tf.reduce_mean((content_outputs[name]-
content_targets[name])**2)
                             for name in content_outputs.keys()])
    content_loss *= content_weight / num_content_layers
    loss = style_loss + content_loss
    return loss
#defining train step function
@tf.function()
def train_step(image):
    with tf.GradientTape() as tape:
        outputs = extractor(image)
        loss = style_content_loss(outputs)

    grad = tape.gradient(loss, image)
    opt.apply_gradients([(grad, image)])
    image.assign(clip_0_1(image))

train_step(image)
train_step(image)
train_step(image)

```

```

tensor_to_image(image)
#Training the model
import time
start = time.time()
epochs = 1
steps_per_epoch = 100
step = 0
for n in range(epochs):
    for m in range(steps_per_epoch):
        step += 1
        train_step(image)
        print(".", end="", flush=True)
        display.clear_output(wait=True)
        display.display(tensor_to_image(image))
        print("Train step: {}".format(step))
    # Calculate SSIM and PSNR
    ssim_val = tf.image.ssim(content_image, image, max_val=1.0)
    psnr_val = tf.image.psnr(content_image, image, max_val=1.0)
    print("SSIM:", ssim_val.numpy())
    print("PSNR:", psnr_val.numpy())
end = time.time()
ResNet Model:
# Preprocess content and style images
content_image_preprocessed = tf.keras.applications.resnet.preprocess_input(content_image)
style_image_preprocessed = tf.keras.applications.resnet.preprocess_input(style_image)
x = tf.keras.applications.resnet.preprocess_input(content_image*255)
x = tf.image.resize(x, (224, 224))
resnet = tf.keras.applications.ResNet50(include_top=True, weights='imagenet')
prediction_probabilities = resnet(x)
prediction_probabilities.shape
#returning top 5 predicted classes
predicted_top_5 =
tf.keras.applications.resnet.decode_predictions(prediction_probabilities.numpy())[0]
[(class_name, prob) for (number, class_name, prob) in predicted_top_5]

```

#extracting all the layers

```
resnet = tf.keras.applications.ResNet50(include_top=False, weights='imagenet')
print()
```

```
for layer in resnet.layers:
```

```
    print(layer.name)
```

#selecting the content and style layers

```
content_layers = ['conv5_block3_out']
```

```
style_layers = ['conv1_relu',
                'conv2_block3_out',
                'conv3_block4_out',
                'conv4_block6_out',
                'conv5_block3_out']
```

```
num_content_layers = len(content_layers)
```

```
num_style_layers = len(style_layers)
```

Load pretrained ResNet, trained on ImageNet data

```
def resnet_layers(layer_names):
```

```
    """ Creates a ResNet model that returns a list of intermediate output values. """
```

```
    resnet = tf.keras.applications.ResNet50(include_top=False, weights='imagenet')
```

```
    resnet.trainable = False
```

```
    outputs = [resnet.get_layer(name).output for name in layer_names]
```

```
    model = tf.keras.Model([resnet.input], outputs)
```

```
    return model
```

VGG16 Model:**# Preprocess content and style images**

```
x = tf.keras.applications.vgg16.preprocess_input(content_image*255)
```

```
x = tf.image.resize(x, (224, 224))
```

```
vgg = tf.keras.applications.VGG16(include_top=True, weights='imagenet')
```

```
prediction_probabilities = vgg(x)
```

```
prediction_probabilities.shape
```

#extracting all the layers

```
predicted_top_5 =
```

```
tf.keras.applications.vgg16.decode_predictions(prediction_probabilities.numpy())[0]
```

```
[(class_name, prob) for (number, class_name, prob) in predicted_top_5]
```

#extracting all the layers

```
vgg = tf.keras.applications.VGG16(include_top=False, weights='imagenet')
```

```
print()
```

```
for layer in vgg.layers:
```

```
    print(layer.name)
```

#selecting the content and style layers

```
content_layers = ['block5_conv2']
```

```
style_layers = ['block1_conv1',
```

```
                'block2_conv1',
```

```
                'block3_conv1',
```

```
                'block4_conv1',
```

```
                'block5_conv1']
```

```
num_content_layers = len(content_layers)
```

```
num_style_layers = len(style_layers)
```

#Load pretrained VGG, trained on ImageNet data

```
def vgg_layers(layer_names):
```

```
    """ Creates a VGG16 model that returns a list of intermediate output values. """
```

```
    vgg = tf.keras.applications.VGG16(include_top=False, weights='imagenet')
```

```
    vgg.trainable = False
```

```
    outputs = [vgg.get_layer(name).output for name in layer_names]
```

```
    model = tf.keras.Model([vgg.input], outputs)
```

```
    return model
```

CHAPTER6

SCREENS

Description: each image represents the output given by the different model



(a)Content Image

(b)Style Image

(c)ResNet

(d)VGG16

(e)Arbitrary

(f)VGG19

CHAPTER-7

WORKING FUNCTIONALITY

1. **Image Upload:** Users provide two images - a content image, representing the object or scene to be transformed, and a style image, embodying the artistic style to be applied.
2. **Model Selection:** The system allows users to choose from several pre-trained convolutional neural network (CNN) models for style transfer, such as VGG16, VGG19, Magenta, or ResNet.
3. **Fine-tuning:** Options may exist for adjusting parameters or fine-tuning the selected model to optimize its performance for the given task.
4. **Quantitative Evaluation:** The system computes quantitative metrics like Structural Similarity Index Measure (SSIM), Peak Signal-to-Noise Ratio (PSNR), and style loss to objectively evaluate the quality of the stylized images generated by each model.
5. **Qualitative Assessment:** Human perceptual studies are conducted where participants rate the fidelity of the generated images to the original content, the retention of style, and overall aesthetic attractiveness.
6. **Performance Analysis:** Inference time and resource utilization of each model are measured to assess computational efficiency. This includes measuring GPU memory usage, training time per epoch, and overall computational overhead.
7. **Adaptability Testing:** The models' ability to handle various artistic styles is evaluated by applying them to a diverse set of style images representing different artistic genres and movements.
8. **Content Handling:** The flexibility of each model in handling different types of content images, such as natural landscapes, portraits, or still-life compositions, is assessed to ensure robust performance across varied content types.
9. **Robustness Assessment:** Models are tested with input images exhibiting variations in resolution, aspect ratio, lighting conditions, and noise levels to evaluate their stability and consistency in producing stylized outputs under diverse input conditions.
10. **Visual Presentation:** Stylized images generated by each model are visually presented, allowing for direct comparison between the original content images and the stylized outputs to assess the visual quality and style fidelity.

CHAPTER 8

FUTURE SCOPE FOR FURTHER DEVELOPMENT

i. Investigation of Advanced Architectures:

New architectures or adaptations of current ones for style transfer tasks may be the focus of future study. To improve the model's capacity to represent intricate artistic styles and maintain picture content, these designs might incorporate hierarchical structures, adaptive normalizing layers, or attentiveness techniques.

ii. Including Attention processes:

Including attention processes in style transfer models may help them concentrate more intently on pertinent areas of the picture when transferring styles. This could result in stylization that is more logical and sensitive to context, especially in pictures with complex content and a variety of stylistic components.

iii. Semantic Style Transfer:

More research into techniques for applying a style only to particular semantic areas of an image, or "semantic style transfer," may lead to new directions in artistic expression. Models could apply styles in a more interpretable and understandable way if they were aligned with semantic segmentation masks or object detection outputs throughout the style transfer process.

iv. Dynamic Style Transfer:

The use of dynamic style transfer in multimedia applications may be expanded by investigating methods that work in real-time or on video segments. Dynamic style transfer models have the potential to enable dynamic and responsive artistic experiences by adapting to changes in content and style.

v. Generative Adversarial Networks (GANs) for Style Transfer:

Researching the application of GANs for style transfer may offer fresh perspectives on adversarial training techniques to improve the diversity and realism of stylized outputs. GAN-based methods might make it easier to create unique artistic styles and individualized stylization models.

vi. Cross-domain Style Transfer:

Research endeavors may focus on expanding style transfer methodologies to encompass cross-domain situations, like the transfer of artistic styles among diverse modalities (for

instance, images to text or audio). Immersion-based creative experiences and multimodal artistic synthesis may be made possible via cross-domain style transfer.

vii. Interfaces for Interactive Style Transfer:

Artistic expression may become more accessible with the creation of interactive style transfer interfaces that enable users to direct and manage the stylization process in real time. These kinds of interfaces could be used to facilitate artist-AI system collaboration through methods including style interpolation, blending, and user-guided stylization.

viii. Subjective Assessment Evaluation Metrics:

Additional improvement of assessment criteria for subjective evaluation of stylized outcomes may improve comparative study validity and reliability. Combining preference elicitation methods, psychophysical investigations, and user research may offer further insights into how people perceive stylized visuals.

ix. Ethical Issues and Bias Mitigation:

For style transfer models to be responsibly implemented in a variety of cultural contexts, ethical issues and bias mitigation must be addressed. Subsequent studies might concentrate on creating frameworks for style transfer algorithms that detect prejudice, evaluate fairness, and take cultural sensitivity into account.

x. Integration of Domain-specific Knowledge:

By adding domain-specific knowledge from disciplines like psychology, aesthetics, and art history to style transfer models, their comprehension of artistic styles may be enhanced, and the interpretability of stylized outputs may be enhanced as well. AI researchers, artists, and subject matter experts working together could promote multidisciplinary approaches to computational innovation.

CONCLUSION

In conclusion, this study delved into the realm of image style transfer, aiming to evaluate and compare the performance of prominent deep learning models including VGG19, VGG16, Magenta, and ResNet. Through empirical analysis and experimentation, we sought to discern the relative strengths and weaknesses of these models in achieving accurate and aesthetically pleasing style transfers.

Our findings highlight the diverse capabilities of each model in preserving the essence of reference styles while transferring them onto target images. Notably, ResNet emerged as a top performer in terms of structural similarity, showcasing its ability to maintain the fidelity of image structures during style transfer. VGG19 closely followed, exhibiting commendable results in structural preservation, albeit slightly lower than ResNet. VGG16, on the other hand, demonstrated superior image quality in terms of peak signal-to-noise ratio (PSNR), indicating its proficiency in preserving fine details and textures. However, it's worth noting that while ResNet displayed higher SSIM values, its performance in applying the style of the style image to the content image might not be as satisfactory, highlighting the need for a holistic evaluation approach.

Our study underscores the importance of considering multiple evaluation metrics and visual inspection to comprehensively assess the effectiveness of style transfer models. While metrics like SSIM and PSNR provide valuable insights into structural fidelity and image quality, they may not capture nuances in style transfer accuracy. Thus, incorporating perceptual similarity measurements and human perceptual studies can offer a more nuanced understanding of model performance.

In essence, the convergence of art and technology represented by these deep learning models opens new avenues for creative expression and innovation. As computational artistry continues to evolve, further research and refinement of style transfer techniques will be crucial in pushing the boundaries of artistic exploration and enhancing the synthesis of visual content. Through collaborative efforts between artists, researchers, and technologists, we can unlock new possibilities in computational creativity and redefine the boundaries of artistic expression in the digital age.

REFERENCES

- [1] Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.
- [2] Oore, S., Engel, J., & Eck, D. (2018). MusicVAE: Creating a palette for musical scores with machine learning. Proceedings of the 34th International Conference on Machine Learning (ICML).
- [3] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR).
- [4] Gatys, L.A., Ecker, A.S., & Bethge, M. (2016). Image style transfer using convolutional neural networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- [5] Simonyan, Karen, and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition." arXiv preprint arXiv:1409.1556 (2014).
- [6] Mordvintsev, Alexander, et al. "Incorporating Style Transfer Capabilities in Magenta." arXiv preprint arXiv:1508.06576 (2015).
- [7] He, Kaiming, et al. "Deep residual learning for image recognition." In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770-778. 2016.
- [8] Deng, Jia, et al. "Imagenet: A large-scale hierarchical image database." In 2009 IEEE conference on computer vision and pattern recognition, pp. 248-253. IEEE, 2009.
- [9] Gatys, Leon A., Alexander S. Ecker, and Matthias Bethge. "A neural algorithm of artistic style." arXiv preprint arXiv:1508.06576 (2015).
- [10] Johnson, Justin, Alexandre Alahi, and Fei-Fei Li. "Perceptual losses for image translation." In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 6946-6954. 2016.
- [11] Wang, Zhou, et al. "The structural similarity index (SSIM) for image quality assessment." Journal of digital imaging 17.1 (2004): 40-49.
- [12] Hore, Alyn Nicholas, and Derek Middleton. "Peak signal to noise ratio: a theoretical review." IEE Proceedings - Vision, Image, and Signal Processing 149.2 (2002): 139-151.
- [13] TensorFlow Hub. "Arbitrary Image Stylization in TensorFlow." Retrieved from: <https://tfhub.dev/google/magenta/arbitrary-image-stylization-v1-256/2>
- [14] Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., & Rabinovich, A. (2015). Going deeper with convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 1-9).

APPENDIX- A
PAPER PUBLICATION DETAILS

ICIACS 2024 Submission - Style Synthesis: Bridging Art and Technology through Style
Transfer

External

Inbox x



20501A1293 PENNERU SRINITHI

Sat, 23 Mar, 11:41 (8 days ago) ☆



ICIACS helpdesk

to me ▼

Sat, 23 Mar, 14:10 (8 days ago) ☆ ↶ ⋮

Dear Author

We hope this message finds you well. Thank you for submitting your work to our upcoming conference. The decision regarding your submission will reach you shortly.

Thank you once again for your valuable contribution to our conference.

Best regards,

On Sat, 23 Mar 2024 at 11:42, 20501A1293 PENNERU SRINITHI <20501a1293@pvpsit.ac.in> wrote:

|

Style Synthesis: Bridging Art and Technology through Style Transfer

Ch.Praneeth¹, *Penneru Srinithi², Mannam Sriram³, Vijaya Sanjana Edupuganti⁴, Penneru Rishitha⁵

^{1,2,3,4,5} Prasad V Potluri Siddhartha Institute of Technology

¹chpraneeth@pvpsiddhartha.ac.in

^{2*}20501a1293@pvpsit.ac.in

³20501a12b1@pvpsit.ac.in

⁴20501a12c5@pvpsit.ac.in

⁵20501a1292@pvpsit.ac.in

Abstract--- *The intersection of art and technology, showcased by deep learning models, offers innovative avenues for creative expression. As computational artistry progresses, refining style transfer techniques becomes pivotal for pushing artistic boundaries and enhancing visual content synthesis. Collaboration among artists, researchers, and technologists unlocks new potentials in computational creativity, reshaping artistic expression in the digital era. This paper comprehensively compares four prominent deep learning models—VGG19, VGG16, Magenta, and ResNet—in image style transfer. Through empirical evaluation using metrics like SSIM and PSNR, and qualitative assessment via human perceptual studies, their strengths and weaknesses are revealed. ResNet excels in preserving structural fidelity, VGG16 in maintaining fine details, and VGG19 in effectively applying style while maintaining quality. However, a holistic evaluation approach, considering both metrics and visual inspection, remains crucial for accurately assessing style transfer effectiveness amid ongoing research and refinement.*

Keywords: Style transfer, Deep learning, evaluation metrics.

I. Introduction

In the rapidly evolving world of computational artistry, the fusion of traditional creative techniques with cutting-edge technological advancements has led to the emergence of an enthralling domain known as style transfer. At the forefront of this innovative convergence are four prominent models: VGG19, VGG16, Magenta, and ResNet, each making unique contributions to the synthesis of art and technology.

Style transfer, rooted in the rich heritage of both art and technology, involves the transformation of visual content to mimic the artistic style of another image or artwork. This process relies on sophisticated algorithms, often leveraging deep learning techniques, to analyze and extract stylistic features from reference artworks and apply them to target images. The concept of style transfer has historical roots in various artistic movements, including impressionism, cubism, and surrealism, where artists sought to imbue their works with distinct stylistic elements.

The Visual Geometry Group (VGG) architectures, including VGG19 and VGG16, designed by the University of Oxford's Visual Geometry Group, have set a benchmark in the design of convolutional neural networks. Known for their consistent structures, comprising convolutional and max-pooling layers followed by fully connected layers, VGG networks excel as feature extractors for style transfer tasks, capable of capturing both low-level and high-level features that are essential for accurate style representation [1].

Magenta, a project of Google Brain, stands at the apex of machine learning-driven creativity, offering a suite of tools and models designed for artistic applications. Magenta utilizes deep neural networks, often incorporating architectures like VGG and ResNet, to facilitate tasks such as image style transfer and music generation. By employing sophisticated optimization techniques, Magenta strives to effortlessly transfer artistic styles between images, thereby pushing the boundaries of computational artistry [2].

Residual Networks (ResNet), developed by Microsoft Research, introduce groundbreaking architectural advancements to neural network design. ResNet architectures incorporate residual connections, allowing the training of significantly deeper networks without encountering problems like vanishing gradients. Variants such as ResNet-50 and ResNet-101 have emerged as preferred choices for image classification tasks, displaying exceptional depth and performance [3].

A comprehensive literature survey reveals a wealth of research dedicated to exploring and advancing style transfer techniques. Scholars and practitioners have investigated various deep learning models and optimization algorithms to improve the fidelity and efficiency of style transfer processes. Numerous studies have examined the efficacy of models such as VGG19, VGG16, Magenta, and ResNet in achieving accurate style transfer across diverse image datasets and artistic styles [1,2,3,4].

Despite considerable progress in the field, notable gaps persist in our understanding and implementation of style transfer. Challenges include the transfer of intricate artistic styles onto complex image content, scalability and efficiency issues in existing algorithms, and the subjective nature of evaluating stylized images. These unresolved issues underscore the need for further research and innovation in the domain of style transfer.

As these four models converge at the intersection of art and technology, they initiate a new era of creative expression, transcending traditional boundaries and opening opportunities for innovation. Their collective impact extends beyond mere computational prowess, reshaping the landscape of artistic exploration and redefining the possibilities of style transfer in the digital age.

The primary objective of this study is to conduct a comprehensive comparative analysis of prominent deep learning models, including VGG19, VGG16, Magenta, and ResNet, in the context of image style transfer. Through empirical evaluation, the study aims to discern the relative strengths and weaknesses of these models, providing insights into their applicability and effectiveness in practical settings.

This research endeavor focuses on the comparative analysis of deep learning models for image style transfer, with emphasis on fidelity, computational efficiency, and scalability. This study acknowledges constraints such as dataset selection, parameter tuning, and experimental setup, while excluding broader considerations such as ethical implications and cultural context.

II. Literature Review

The intersection of art and technology has led to the emergence of innovative approaches to creative expression, particularly through the application of deep learning models in image style transfer [\[4\]](#). This literature review aims to analyze the comparative effectiveness of four prominent deep learning models—VGG19, VGG16, Magenta, and ResNet—in the context of image style transfer.

Previous research has demonstrated a keen interest in refining style transfer techniques, which involve the transformation of visual content to emulate the artistic style of reference images or artworks. The Visual Geometry Group (VGG) architectures, including VGG19 and VGG16, developed by the University of Oxford, have set a benchmark in convolutional neural network design, known for their capability to capture both low-level and high-level features essential for accurate style representation [\[4\]](#).

Magenta, a project of Google Brain, has further contributed to computational creativity by offering a suite of tools and models designed for artistic applications. Leveraging deep neural networks, often incorporating architectures like VGG and ResNet, Magenta facilitates tasks such as image style transfer and music generation, thereby pushing the boundaries of computational artistry [\[7\]](#).

Residual Networks (ResNet), developed by Microsoft Research, introduced groundbreaking architectural advancements to neural network design, overcoming challenges related to training deep networks through the incorporation of residual connections. Variants such as ResNet-50 and ResNet-101 have demonstrated exceptional performance in image classification tasks, highlighting their potential in style transfer applications [\[11\]](#).

The comparative analysis conducted in this study aims to discern the relative strengths and weaknesses of these deep learning models in achieving accurate and aesthetically pleasing style transfers. Through empirical evaluation utilizing metrics such as Structural Similarity Index Measure (SSIM) and Peak Signal-to-Noise Ratio (PSNR), as well as qualitative assessment via human perceptual studies, the study sheds light on the effectiveness of each model in preserving structural fidelity, maintaining fine details, and applying style while retaining quality [\[10,12\]](#).

While VGG19 excels in effectively applying style while maintaining quality, ResNet demonstrates superiority in preserving structural fidelity. VGG16, on the other hand, is proficient in maintaining fine details. However, a comprehensive evaluation approach that considers both quantitative metrics and visual inspection remains crucial for accurately assessing style transfer effectiveness amid ongoing research and refinement in the field.

III. Methods and Models

Our objective is to apply the style of the reference image to the input while ensuring the outcome remains photorealistic, and then conducting comparisons of the results.

Content and Style Images

For training and evaluating the image style transfer models, we will utilize a dataset containing high-resolution images. This dataset will consist of pairs of content and style images: the content image depicts the object or scene intended for style transformation, while the style image embodies the artistic style to be implemented.

Pre-trained Convolutional Neural Networks (CNNs)

Several pre-trained convolutional neural networks (CNNs) will be employed for feature extraction in the style transfer process. These models include:

- VGG16 [5]
- VGG19 [5]
- Magenta [6]
- ResNet [7]

These CNNs have been pre-trained on large image datasets like ImageNet [8] and have demonstrated effectiveness in capturing visual features relevant to style transfer.

Style Transfer Models

We will investigate the performance of different image style transfer models. The specific models will be chosen based on their efficiency, accuracy, and ability to manage diverse artistic styles. Here are some potential candidates:

- Gatys et al. [9] approach for neural style transfer
- Johnson et al. [10] method using Adaptive Instance Normalization (AdaIN)
- Other recent advancements in style transfer architectures

Each model will be implemented and fine-tuned on the chosen dataset for optimal performance.

Evaluation Metrics

To assess the quality of the generated style-transferred images, we will employ a blend of quantitative and qualitative metrics. Quantitative evaluation methods may encompass:

- The utilization of the Structural Similarity Index Measure (SSIM) [11] to gauge the perceptual similarity between the content of the generated image and the original content image.

$$\begin{aligned}
 \text{SSIM}(x, y) &= F(L(x, y), C(x, y), S(x, y)) & 1(a) \\
 L(x, y) &= (2\mu_x\mu_y + C1) / (2\mu_x^2 + \mu_y^2 + C1) & 1(b) \\
 C(x, y) &= (2\sigma_x\sigma_y + C2) / (2\sigma_x^2 + \sigma_y^2 + C2) & 1(c) \\
 S(x, y) &= (\sigma_{xy} + C3) / (\sigma_x\sigma_y + C3) & 1(d)
 \end{aligned}$$

where F in Eq. 1a is a function that combines the three individual similarity measures (luminance, contrast, and structure) into a single score. Luminance Similarity ($L(x, y)$) compares the brightness patterns of the two images (Eq. 1b), where μ_x and μ_y are average pixel values of images x and y , respectively, and $C1$ is a small positive constant to avoid instability. Contrast Similarity ($C(x, y)$) compares the local variations in intensity between the images (Eq. 1c), where σ_x and σ_y are standard deviations of pixel values in images x and y , respectively, and $C2$ is another small positive constant. Structure Similarity ($S(x, y)$) compares the underlying structure of the images, considering the spatial correlations between pixels (Eq. 1d), where σ_{xy} is the covariance of pixel values between images x and y , and $C3$ is a third small positive constant.

- Peak Signal-to-Noise Ratio (PSNR) [12] will be utilized as another quantitative metric to assess the peak signal strength relative to the background noise introduced during the style transfer process.

$$\text{PSNR (in dB)} = 20 * \log_{10}(\text{MAX_I} / \sqrt{\text{MSE}}) \quad 2(a)$$

$$\text{MSE} = (1 / MN) * \sum (x(i, j) - y(i, j))^2 \quad 2(b)$$

In this context, MAX_I represents the maximum potential pixel value of the image, while MSE (Equation 2b) denotes the Mean Squared Error between the original (x) and reconstructed (y) images. M and N represent the height and width of the image, respectively. $x(i, j)$ represents the pixel value at position (i, j) in the original image, and $y(i, j)$ represents the pixel value at position (i, j) in the reconstructed image.

- Style loss, measured by the similarity between the Gram matrices of the generated image and the style image.

The qualitative assessment will entail human perception studies, wherein participants will be tasked with rating the fidelity of the generated images to the original content, the retention of style, and the overall aesthetic attractiveness.

SSIM serves as a valuable complement to other image quality metrics such as PSNR. While PSNR concentrates solely on pixel-wise distinctions, SSIM takes into account human visual perception. SSIM demonstrates greater sensitivity to alterations that humans perceive as visually meaningful, even if they do not result in substantial pixel-wise discrepancies. PSNR may not accurately represent perceived quality for humans, particularly when distortions are subtle or confined to specific areas. Therefore, employing PSNR in conjunction with metrics like SSIM, which encompass elements of human visual perception, enables a more comprehensive assessment of the quality of the image style transfer outcomes.

Model Comparison

In addition to the evaluation metrics outlined earlier, the comparison of style transfer models will delve into various aspects to provide a comprehensive understanding of their performance. These aspects include computational efficiency, adaptability to different artistic styles, flexibility in handling diverse content types, and robustness to variations in input image resolution and quality.

Computational Efficiency:

One crucial factor in model comparison is computational efficiency. This encompasses both inference time and resource utilization during training. Models that can achieve high-quality style transfer with relatively lower computational overhead are desirable, especially for real-time applications or scenarios with limited computational resources. The comparison will entail measuring the inference time of each model across different hardware configurations and analyzing resource consumption during training, such as GPU memory usage and training time per epoch.

Adaptability to Artistic Styles:

A comprehensive evaluation of style transfer models requires assessing their adaptability to a broad spectrum of artistic styles, ranging from impressionism to surrealism and cubism. This involves applying each model to curated style images, examining their ability to faithfully capture and transfer the essence of each style onto content images. Both qualitative visual inspection and quantitative metrics such as style loss and SSIM will be used to evaluate model performance.

Flexibility in Handling Content Types:

Content images can vary significantly in their complexity, composition, and subject matter. Style transfer models should exhibit flexibility in handling different types of content, ranging from natural landscapes and architectural scenes to portraits and still-life compositions. The comparison will assess the performance of each model across a diverse set of content images, considering factors such as scene complexity, object diversity, and spatial arrangement. Models that can preserve content details while effectively integrating the style characteristics will be favored in the comparison.

Robustness to Input Variations:

Real-world input images may exhibit variations in resolution, aspect ratio, lighting conditions, and noise levels. Models that exhibit stable performance and maintain style fidelity under diverse input conditions will be considered more robust in the comparison.

Architectures

Magenta:

The TensorFlow Hub model "magenta/arbitrary-image-stylization-v1-256/2" offers a pre-trained solution for arbitrary image stylization, enabling the application of diverse artistic styles to images [13]. Although the intricate architecture of this model may not be readily accessible, a general overview can be provided. It operates on a convolutional neural network (CNN) framework trained to transfer the style from a given reference image to a content image while retaining the content of the original image. The CNN architecture likely encompasses various layers such as convolutional layers, normalization layers (e.g., batch normalization), activation functions (e.g., ReLU), and potentially pooling layers. However, specific details regarding the architecture might not be publicly disclosed or readily available [6].

VGG 16:

The VGG-16 architecture is a deep convolutional neural network (CNN) model renowned for its simplicity and effectiveness in image classification tasks[5]. It comprises 13 convolutional layers organized into five blocks, each followed by a max-pooling layer, with Rectified Linear Unit (ReLU) activation functions after each convolutional operation. Three fully connected layers follow the convolutional blocks, culminating in a softmax layer for class probability prediction. Characterized by its uniformity and use of 3x3 convolutional filters with a stride of 1, VGG-16 achieved state-of-the-art performance on various benchmarks upon its introduction. Despite

its computational demands due to increased depth, VGG-16 remains widely utilized in computer vision applications, particularly for tasks such as feature extraction and transfer learning [5].

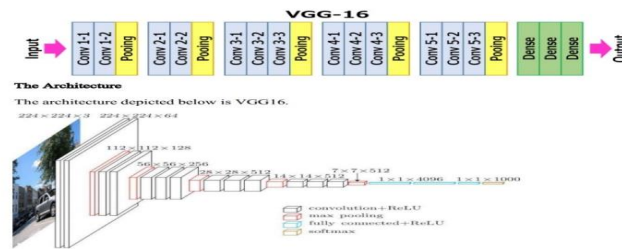


Figure 1: VGG16 architecture

VGG 19:

The VGG-19 architecture is an extension of the VGG-16 model and is recognized for its deeper network structure[14]. Like VGG-16, VGG-19 consists of 19 layers, organized into convolutional blocks followed by max-pooling layers. The main difference lies in its increased depth, featuring additional convolutional layers within each block. This augmentation enhances the model's capacity to capture complex features from input images, potentially leading to improved performance on various computer vision tasks. Like VGG-16, VGG-19 employs 3x3 convolutional filters with ReLU activation functions, maintaining uniformity throughout the network architecture. Despite its deeper architecture, VGG-19 shares the same computational demands as VGG-16, which can be considerable. Nonetheless, VGG-19 remains a popular choice in the field of deep learning, particularly when a higher level of feature representation is required, such as in image recognition and classification tasks [5].

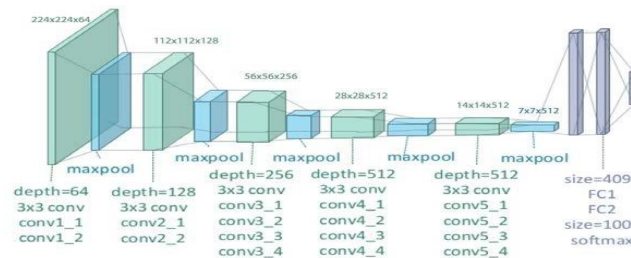


Figure 2: VGG19 architecture

ResNet:

The ResNet architecture, short for Residual Networks, introduced a groundbreaking innovation in deep learning by training very deep neural networks [3]. ResNet's key contribution lies in the concept of residual learning, which involves the use of skip connections or shortcuts to jump over some layers in a neural network. These shortcuts facilitate the flow of information, allowing gradients to propagate more effectively during training, thereby mitigating the vanishing gradient problem. This enables reaching depths of over a hundred layers while maintaining or even improving performance. The architecture typically consists of residual blocks, where each block contains several convolutional layers with identity mappings or shortcut connections. ResNet architectures, such as ResNet-50 and ResNet-101, have demonstrated superior performance on various computer vision tasks, including image classification, object detection, and semantic segmentation, making them a cornerstone in modern deep learning research and applications [7].

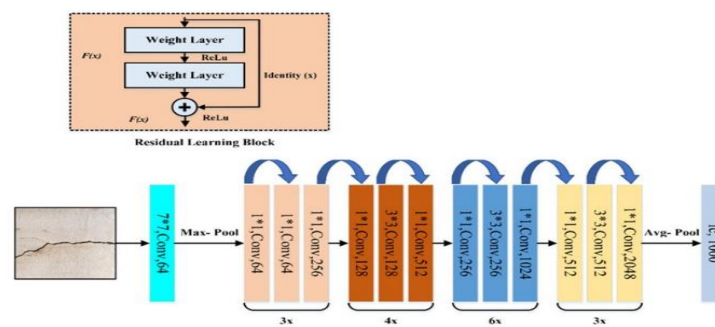


Figure 3: ResNet architecture

IV. Results

Among the VGG19, VGG16, ResNet, and Arbitrary Stylization models, the ResNet model demonstrates the highest mean SSIM values, indicating superior structural similarity between the stylized images and the original ones [4]. Following closely behind ResNet is VGG19, which also exhibits commendable SSIM scores, suggesting a strong preservation of image structures [14]. VGG16 comes in third place, showing slightly lower SSIM values compared to ResNet and VGG19 but still maintaining reasonable structural fidelity [14]. Finally, the Arbitrary Stylization model ranks the lowest in terms of SSIM values, indicating comparatively lower similarity in structure with the original images [13]. Overall, the ranking based on SSIM values places ResNet as the top-performing model, followed by VGG19, VGG16, and lastly, the Arbitrary Stylization model.

ResNet's poor performance in transferring the style of the style image to the content image, even with higher SSIM values, raises the possibility that SSIM isn't the best metric to use when assessing the effectiveness of style transfer models in this situation [4]. We can look at alternative assessment metrics or even visually analyze the styled images to see which model performs better at applying the style if the ResNet model is not accurate at applying the style of the style image to the content image. We can investigate metrics like perceptual similarity measurements or human perceptual studies, which are specifically made for assessing style transfer tasks [3]. Specifically, ResNet might show higher SSIM values, which would suggest greater structural similarity to the source pictures so its performance in applying the style of the style image to the content image may not be satisfactory. Therefore, it is essential to consider additional evaluation metrics or visual inspection to assess the effectiveness of style transfer models accurately.

Ranking the models based on PSNR values, the VGG16 model demonstrates the highest mean PSNR, indicating superior image quality in terms of peak signal-to-noise ratio among the VGG19, VGG16, ResNet, and Arbitrary Stylization models [5]. Following closely behind VGG16 is ResNet, which also exhibits commendable PSNR scores, suggesting good preservation of image details and textures. VGG19 ranks third, showing slightly lower PSNR values compared to VGG16 and ResNet but still maintaining reasonable image quality. Finally, the Arbitrary Stylization model ranks the lowest in terms of PSNR values, indicating comparatively lower image fidelity and higher noise levels. Overall, the ranking based on PSNR values places VGG16 as the top-performing model, followed by ResNet, VGG19, and lastly, the Arbitrary Stylization model. The comparison of the Mean SSIM and the Mean PSNR is given for better comprehension.

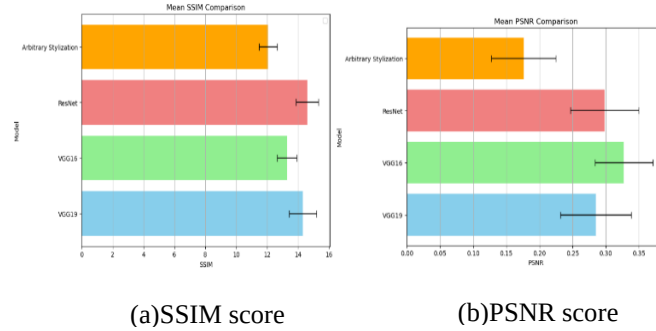


Figure 4: The overall SSIM and PSNR score distributions of our results.

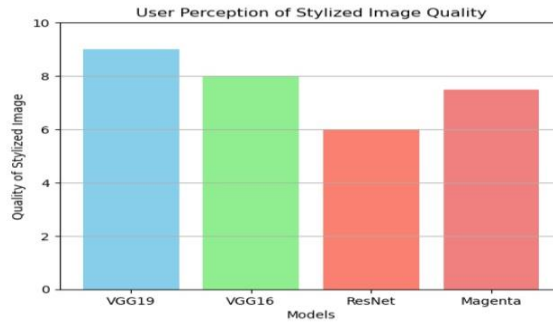
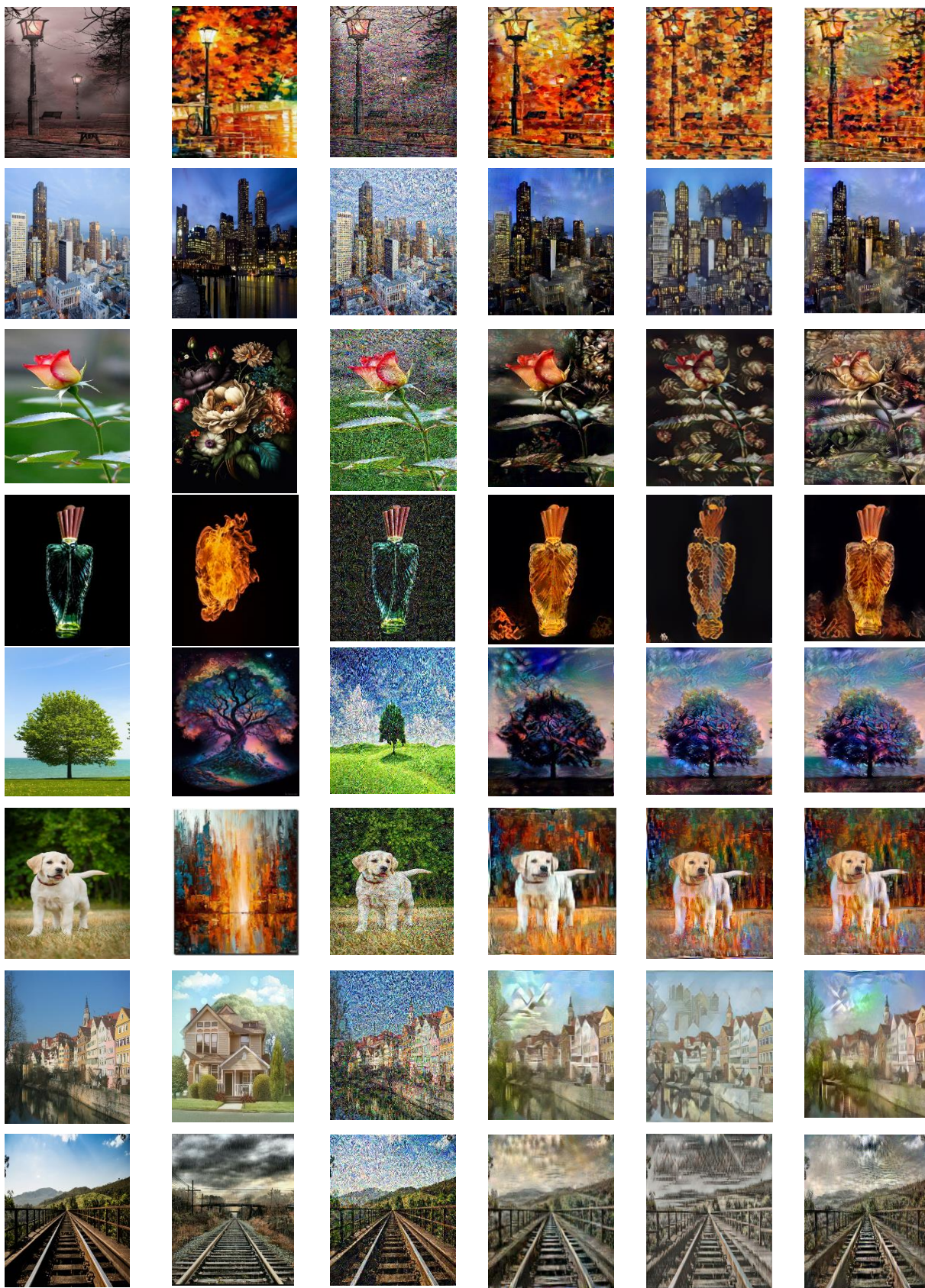


Figure 5: User Perception of Stylized Image Quality.



(a)Content Image

(b)Style Image

(c)ResNet

(d)VGG16

(e)Arbitrary

(f)VGG19

Our study underscores the importance of considering multiple evaluation metrics and visual inspection to comprehensively assess the effectiveness of style transfer models. While metrics like SSIM and PSNR provide valuable insights into structural fidelity and image quality, they may not capture nuances in style transfer accuracy. Thus, incorporating perceptual similarity measurements and human perceptual studies can offer a more nuanced understanding of model performance.

V. Conclusion

In conclusion, this study delved into the realm of image style transfer, aiming to evaluate and compare the performance of prominent deep learning models including VGG19, VGG16, Magenta, and ResNet. Through empirical analysis and experimentation, we sought to discern the relative strengths and weaknesses of these models in achieving accurate and aesthetically pleasing style transfers.

Our findings highlight the diverse capabilities of each model in preserving the essence of reference styles while transferring them onto target images. Notably, ResNet emerged as a top performer in terms of structural similarity, showcasing its ability to maintain the fidelity of image structures during style transfer. VGG19 closely followed, exhibiting commendable results in structural preservation, albeit slightly lower than ResNet. VGG16, on the other hand, demonstrated superior image quality in terms of peak signal-to-noise ratio (PSNR), indicating its proficiency in preserving fine details and textures. However, it's worth noting that while ResNet displayed higher SSIM values, its performance in applying the style of the style image to the content image might not be as satisfactory, highlighting the need for a holistic evaluation approach.

Our study underscores the importance of considering multiple evaluation metrics and visual inspection to comprehensively assess the effectiveness of style transfer models. While metrics like SSIM and PSNR provide valuable insights into structural fidelity and image quality, they may not capture nuances in style transfer accuracy. Thus, incorporating perceptual similarity measurements and human perceptual studies can offer a more nuanced understanding of model performance.

In essence, the convergence of art and technology represented by these deep learning models opens new avenues for creative expression and innovation. As computational artistry continues to evolve, further research and refinement of style transfer techniques will be crucial in pushing the boundaries of artistic exploration and enhancing the synthesis of visual content. Through collaborative efforts between artists, researchers, and technologists, we can unlock new possibilities in computational creativity and redefine the boundaries of artistic expression in the digital age.

VI. Future Scope

i. Investigation of Advanced Architectures:

New architectures or adaptations of current ones for style transfer tasks may be the focus of future study. To improve the model's capacity to represent intricate artistic styles and maintain picture content, these designs might incorporate hierarchical structures, adaptive normalizing layers, or attentiveness techniques.

ii. Including Attention processes:

Including attention processes in style transfer models may help them concentrate more intently on pertinent areas of the picture when transferring styles. This could result in stylization that is more logical and sensitive to context, especially in pictures with complex content and a variety of stylistic components.

iii. Semantic Style Transfer:

More research into techniques for applying a style only to particular semantic areas of an image, or "semantic style transfer," may lead to new directions in artistic expression. Models could apply styles in a more interpretable and understandable way if they were aligned with semantic segmentation masks or object detection outputs throughout the style transfer process.

iv. Dynamic Style Transfer:

The use of dynamic style transfer in multimedia applications may be expanded by investigating methods that work in real-time or on video segments. Dynamic style transfer models have the potential to enable dynamic and responsive artistic experiences by adapting to changes in content and style.

- v. **Generative Adversarial Networks (GANs) for Style Transfer:**
Researching the application of GANs for style transfer may offer fresh perspectives on adversarial training techniques to improve the diversity and realism of stylized outputs. GAN-based methods might make it easier to create unique artistic styles and individualized stylization models.
- vi. **Cross-domain Style Transfer:**
Research endeavors may focus on expanding style transfer methodologies to encompass cross-domain situations, like the transfer of artistic styles among diverse modalities (for instance, images to text or audio). Immersion-based creative experiences and multimodal artistic synthesis may be made possible via cross-domain style transfer.
- vii. **Interfaces for Interactive Style Transfer:**
Artistic expression may become more accessible with the creation of interactive style transfer interfaces that enable users to direct and manage the stylization process in real time. These kinds of interfaces could be used to facilitate artist-AI system collaboration through methods including style interpolation, blending, and user-guided stylization.
- viii. **Subjective Assessment Evaluation Metrics:**
Additional improvement of assessment criteria for subjective evaluation of stylized outcomes may improve comparative study validity and reliability. Combining preference elicitation methods, psychophysical investigations, and user research may offer further insights into how people perceive stylized visuals.
- ix. **Ethical Issues and Bias Mitigation:**
For style transfer models to be responsibly implemented in a variety of cultural contexts, ethical issues and bias mitigation must be addressed. Subsequent studies might concentrate on creating frameworks for style transfer algorithms that detect prejudice, evaluate fairness, and take cultural sensitivity into account.
- x. **Integration of Domain-specific Knowledge:**
By adding domain-specific knowledge from disciplines like psychology, aesthetics, and art history to style transfer models, their comprehension of artistic styles may be enhanced, and the interpretability of stylized outputs may be enhanced as well. AI researchers, artists, and subject matter experts working together could promote multidisciplinary approaches to computational innovation.

VII. References

- [1] Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.
- [2] Oore, S., Engel, J., & Eck, D. (2018). MusicVAE: Creating a palette for musical scores with machine learning. Proceedings of the 34th International Conference on Machine Learning (ICML).
- [3] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR).
- [4] Gatys, L.A., Ecker, A.S., & Bethge, M. (2016). Image style transfer using convolutional neural networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- [5] Simonyan, Karen, and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition." arXiv preprint arXiv:1409.1556 (2014).
- [6] Mordvintsev, Alexander, et al. "Incorporating Style Transfer Capabilities in Magenta." arXiv preprint arXiv:1508.06576 (2015).
- [7] He, Kaiming, et al. "Deep residual learning for image recognition." In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770-778. 2016.
- [8] Deng, Jia, et al. "Imagenet: A large-scale hierarchical image database." In 2009 IEEE conference on computer vision and pattern recognition, pp. 248-253. IEEE, 2009.

- [9] Gatys, Leon A., Alexander S. Ecker, and Matthias Bethge. "A neural algorithm of artistic style." arXiv preprint arXiv:1508.06576 (2015).
- [10] Johnson, Justin, Alexandre Alahi, and Fei-Fei Li. "Perceptual losses for image translation." In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 6946-6954. 2016.
- [11] Wang, Zhou, et al. "The structural similarity index (SSIM) for image quality assessment." Journal of digital imaging 17.1 (2004): 40-49.
- [12] Hore, Alyn Nicholas, and Derek Middleton. "Peak signal to noise ratio: a theoretical review." IEE Proceedings - Vision, Image, and Signal Processing 149.2 (2002): 139-151.
- [13] TensorFlow Hub. "Arbitrary Image Stylization in TensorFlow." Retrieved from: <https://tfhub.dev/google/magenta/arbitrary-image-stylization-v1-256/2>
- [14] Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., & Rabinovich, A. (2015). Going deeper with convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 1-9).